# Image Compression

# Outline

- Introduction
- Fundamentals
  - Coding, interpixel and psychovisual redundancies
  - Fidelity criteria
- Image compression model
  - Source encoder and decoder
  - Channel encoder and decoder
- Lossless compression
  - Huffman encoding
  - Lossless predictive coding
- Lossy compression
  - Lossy predictive coding
  - Transform coding
  - JPEG

# Introduction

1. Enormous amount of information is stored and transmitted digitally every day.
2. Since much of this on-line information is graphical or pictorial in nature, the storage and communication requirements are immense.
3. Image compression becomes important when either large storage or heavy communication is required.
4. Image compression reduces the amount of data required to represent the information (removal of redundant data).
5. There are two types: information preserving (lossless, error free) and lossy.

# Introduction

6. *Information preserving (Lossless)*: allows an image to be compressed and decompressed without losing information.
7. *Lossy*: provides higher levels of data reduction but result in a less than perfect reproduction of the original image.

# Fundamentals

1. Data are the means by which information is conveyed.
2. Redundant data contains words or data that either provide no relevant information or simply restate that which is already known.
3. Let $n_1$ and $n_2$ be the numbers of information-carrying units in two datasets ($X_1$ and $X_2$) that represent approx. the same amount of information respectively.
4. Given dataset $X_2$, the *relative data redundancy* $R_D$ of the dataset $X_1$ is given by

$$R_D = 1 - \frac{1}{C_R} = \frac{n_1 - n_2}{n_1}$$

where $C_R = \frac{n_1}{n_2}$, $C_R \in [0, \infty)$ and $R_D \in (-\infty, 1]$

Compression ratio

# Fundamentals

5. Case 1: $n_2=n_1$, $C_R=1$ and $R_D=0$. It indicates that, relative to the dataset $X_1$, the representation of information contains no redundant data.
6. Case 2: $n_2<<n_1$, $C_R\rightarrow\infty$ and $R_D\rightarrow 1$. It indicates significant compression and highly redundant data.
7. Case 3: $n_2>>n_1$, $C_R\rightarrow 0$ and $R_D\rightarrow -\infty$. It indicates that the second dataset $X_2$ contains much more data than the original representation.
8. In digital image compression, there are 3 basic types of data redundancy.
   a. coding redundancy in symbol coding
   b. interpixel redundancy between pixels
   c. psychovisual redundancy in visual processing

# Coding Redundancy

1. Let $r$ be a discrete random variable in the interval [0,1].
2. Assume that each $r_k$ occurs with probability $p_r(r_k)$, where

$$p_r\left(r_k\right)=\frac{n_k}{n}, \quad k=0,1,2,...,L-1$$

3. $L$ = number of grey levels
4. $n_k$ = number of times that the $k^{\text{th}}$ grey level appears in an image. Related to occurrence.
5. $n$ = total number of pixels in the image.
6. If the number of bits used to represent $r_k$ is $l(r_k)$, then the average (expected) number of bits is

$$L_{avg}=\sum_{k=0}^{L-1} l\left(r_k\right) p_r\left(r_k\right)$$

# Coding Redundancy: example

1. An 8-level ($L=8$) image has the grey level distribution below.

| $r_k$ | $p_r(r_k)$ | Code 1 | $l_1(r_k)$ | Code 2 | $l_2(r_k)$ |
|---|---|---|---|---|---|
| $r_0 = 0$ | 0.19 | 000 | 3 | 11 | 2 |
| $r_1 = 1/7$ | 0.25 | 001 | 3 | 01 | 2 |
| $r_2 = 2/7$ | 0.21 | 010 | 3 | 10 | 2 |
| $r_3 = 3/7$ | 0.16 | 011 | 3 | 001 | 3 |
| $r_4 = 4/7$ | 0.08 | 100 | 3 | 0001 | 4 |
| $r_5 = 5/7$ | 0.06 | 101 | 3 | 00001 | 5 |
| $r_6 = 6/7$ | 0.03 | 110 | 3 | 000001 | 6 |
| $r_7 = 1$ | 0.02 | 111 | 3 | 000000 | 6 |
| | | 8 code words | | 8 code words | |

**TABLE 8.1** Example of variable-length coding.

2. *Coding scheme 1 (fixed-length coding)*: A natural 3-bit binary code ("code 1" and $l_1(r_k)$) is used to represent the 8 possible grey levels.

# Coding Redundancy: example

3. The average number of bits for coding scheme 1 is given by

$$L_{avg} = \sum_{k=0}^{8-1} l_1(r_k)\, p_r(r_k)$$

$$\Rightarrow L_{avg} = 3 \text{ bits}$$

4. *Coding scheme 2 (variable-length coding)*: The length of code words is inversely proportional to the grey level probability ("code 2" and $l_2(r_k)$).

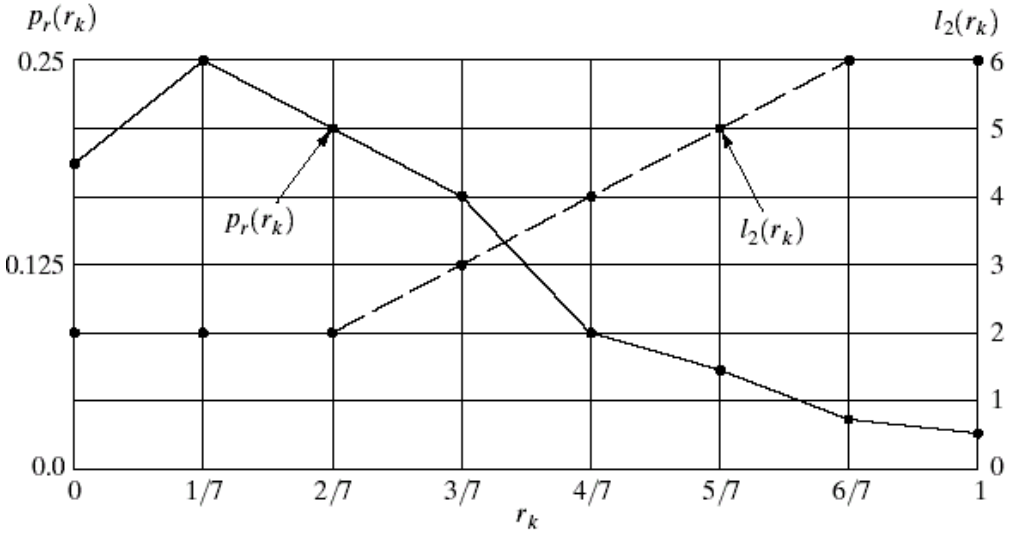



**FIGURE 8.1** Graphic representation of the fundamental basis of data compression through variable-length coding.

Solid = probability
Dash = code word length

The figure is provided by Pearson Education, Digital Image Processing, Gonzalez & Woods, www.ImageProcessingPlace.com

# Coding Redundancy: example

5. The average number of bits for coding scheme 2 is given by

$$L_{avg} = \sum_{k=0}^{8-1} l_2\left(r_k\right) p_r\left(r_k\right)$$

$$\Rightarrow L_{avg} = 2.7 \text{ bits}$$

6. The compression ratio is given by

$$C_R = \frac{n_1}{n_2} = \frac{3}{2.7} = 1.11$$

7. The relative data redundancy is given by

$$R_D = 1 - \frac{1}{C_R} = 1 - \frac{1}{1.11} = 0.099$$

# Coding Redundancy: example

8. Coding scheme 2 = *variable-length coding*: assigning smaller number of bits to the more probable (or more frequently occurring) grey levels than the less probable ones (or less frequently occurring).
9. Coding scheme 2 achieves data compression.
10. Dataset represented by coding scheme 1 contains coding redundancy because it assigns the same number of bits to both the most and least probable grey-level values.

# Interpixel Redundancy

1. The variable-length coding is based on the grey-level histogram and has nothing to do with the correlation (inter-pixel relationship) between pixels.
2. Example: 2 images having the same histogram but different autocorrelation coefficients, which is given by.

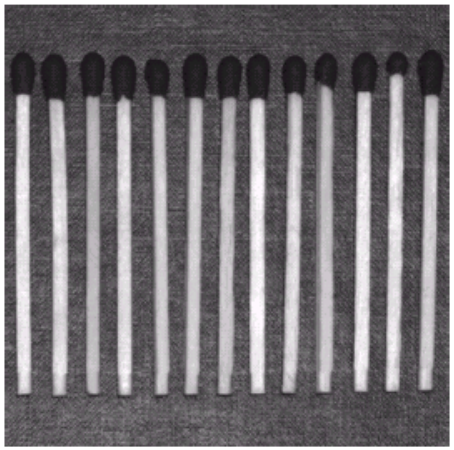
$$\gamma(\Delta n) = \frac{A(\Delta n)}{A(\Delta n = 0)}$$

where $$A(\Delta n) = \frac{1}{N - \Delta n} \sum_{y=0}^{N-1-\Delta n} f(x, y) f(x, y + \Delta n)$$

$\Delta n =$ distance between pixels

$f =$ intensity; $N =$ total number of pixels in an image

Different ways of putting matchsticks

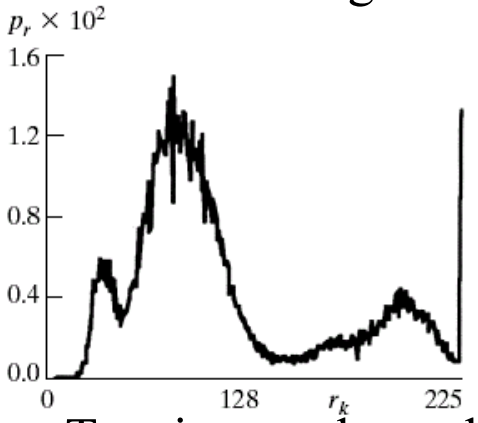


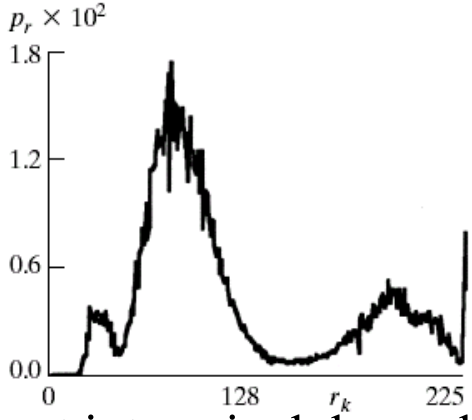
a b
c d
e f

**FIGURE 8.2** Two images and their gray-level histograms and normalized autocorrelation coefficients along one line.

Two image histograms are the same

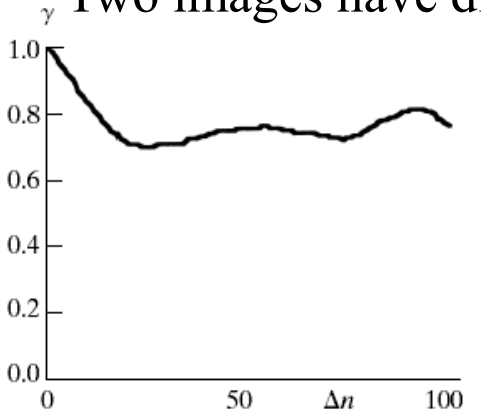
Histograms are the same

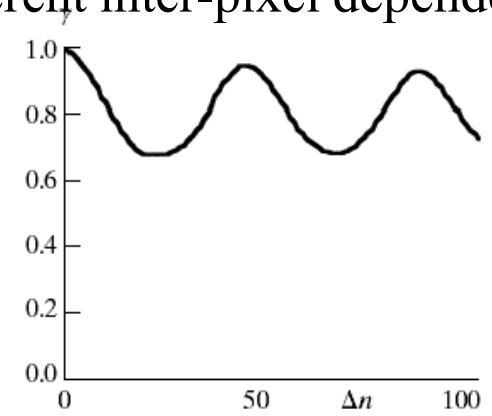



Two images have different inter-pixel dependency

Different inter-pixel dependency/correlation



# Interpixel Redundancy

3. When $\Delta n = 1$, gamma = 0.9922 (Fig. 8.2e) and 0.9928 (Fig. 8.2f). It shows that when the distance between pixels is one, adjacent pixels in both images are *highly correlated*.
4. When $\Delta n = 45$ or $90$ (spacing between the vertical oriented matches) in Fig. 8.2b, pixels are *highly correlated*.
5. *High correlation between pixel intensity values* = adjacent pixel values can be reasonably predicted from the value of current pixel (the concept of interpixel redundancy).
6. To reduce the interpixel redundancies, the 2D image pixel array can be transformed into a more efficient format. It is also called *mapping*.
7. *Reversible mapping*: if the original image elements can be reconstructed from the transformed data set.

# Interpixel Redundancy: An example

1. An electrical assembly drawing example.
2. A binary version (Fig. 8.3a) of the original image (Fig. 8.3b) is obtained based on a global intensity threshold selected in Fig. 8.3c.
3. Because the binary image contains many regions of constant intensity, a more efficient representation can be constructed by mapping the pixels along each scan line ( $f$ = intensity)

$$f(x,0), f(x,1), ..., f(x,N-1)$$

into a sequence of pairs

$$(g_1, w_1), (g_2, w_2), ...$$

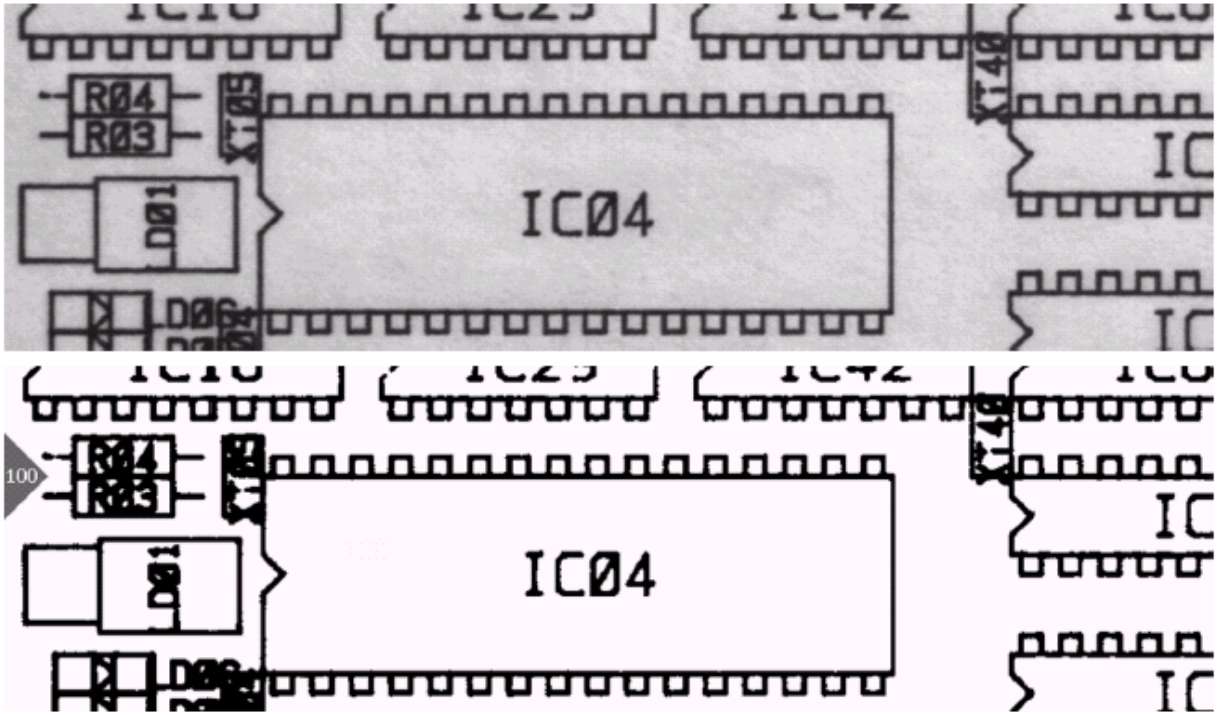
in which

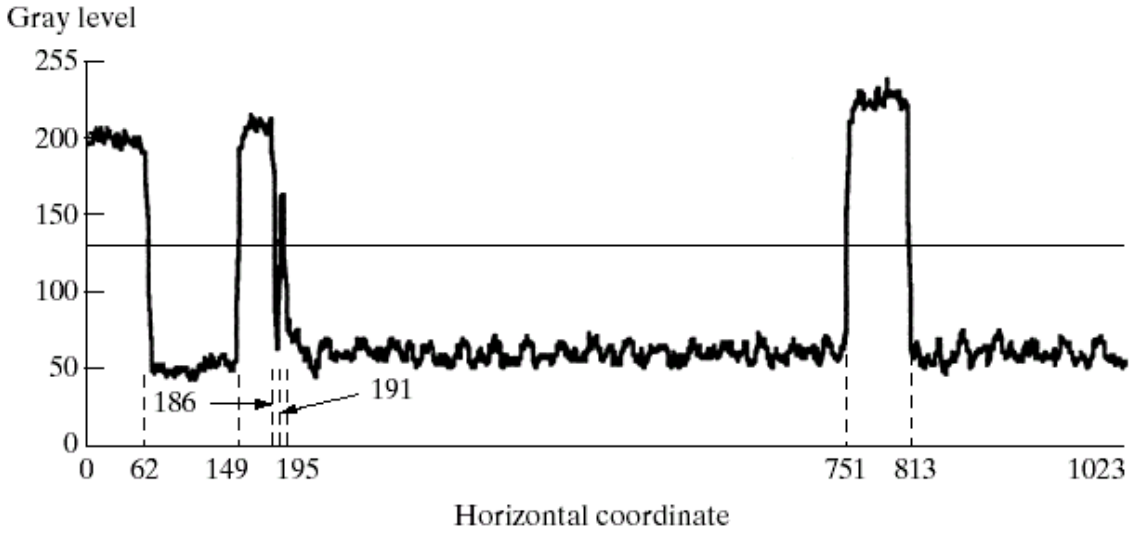
$g_i$ = $i^{\text{th}}$ grey level encountered along the scan line

$w_i$ = run length

Original image

Binary image




a
b
c
d

**FIGURE 8.3** Illustration of run-length coding: (a) original image. (b) Binary image with line 100 marked. (c) Line profile and binarization threshold. (d) Run-length code.

Line 100: (1, 63) (0, 87) (1, 37) (0, 5) (1, 4) (0, 556) (1, 62) (0, 210)

# Interpixel Redundancy: example

4. For line 100: (1,63)(0,87)(1,37)(0,5)(1,4)(0,556)(1,62)(0,210), which needs 1 bit for $g_i$ and 10 bits for $w_i$ (11 bits for each pair). As such, 88 bits are needed to represent a line instead of 1024 bits of binary data. 10 bits can represent numbers between 0 and 1023.
5. For the entire image with 343 lines, there are 1024 X 343 bits, which can be encoded using 12166 pairs (11 bits for each pair).
6. The compression ratio and relative redundancy are given by

$$C_R = \frac{n_1}{n_2} = \frac{(1024)(343)(1)}{(12166)(11)} = 2.63$$

$$R_D = 1 - \frac{1}{C_R} = 1 - \frac{1}{2.63} = 0.62$$

# Psychovisual Redundancy

1. Certain information simply has less relative importance than other information in normal visual processing. This information is called *psychovisual redundancy*.
2. Example: an observer is sensitive to distinguishing features such as edges (sudden change in intensity values) or textural regions (patterns), which help the observer to form an image mentally. On the other hand, the observer in general *does not analyse the quantitative information in the image*.
3. Psychovisual redundancy is associated with real or quantifiable visual information. *Its elimination is possible only because the information itself is not essential for normal visual processing*, e.g., slight reduction in image resolution (irreversible process).

# Psychovisual Redundancy

4. Quantization: maps a broad range of intensity values into a limited number of output values.

a b c

**FIGURE 8.4**
(a) Original image. (b) Uniform quantization to 16 levels. (c) IGS quantization to 16 levels.


Pay attention to the surface of the brush holder

The figure is provided by Pearson Education, Digital Image Processing, Gonzalez & Woods, www.ImageProcessingPlace.com

# Psychovisual Redundancy

5. Fig. 8.4b shows a result of quantization from 256 levels (8 bits) to 16 levels (4 bits). Compression ratio = 2:1.
6. All edges are shown. However, false contours are also shown, and surfaces are rough.
7. Improved grey-scale (IGS) quantization is used to compensate for the visual impact of quantization by adding to each pixel a pseudo-random number, which is generated from the low-order bits of neighbouring pixels (Sum), before quantizing the result.

| Pixel | Gray Level | Sum | IGS Code |
|---|---|---|---|
| $i-1$ | N/A | 0000 0000 | N/A |
| $i$ | 0110 1100 | 0110 1100 | 0110 |
| $i+1$ | 1000 1011 | 1001 0111 | 1001 |
| $i+2$ | 1000 0111 | 1000 1110 | 1000 |
| $i+3$ | 1111 0100 | 1111 0100 | 1111 |

**TABLE 8.2**
IGS quantization procedure.

Last 4 bits are used as pseudo-random number.
Example: Bit-plane representation
(original image)

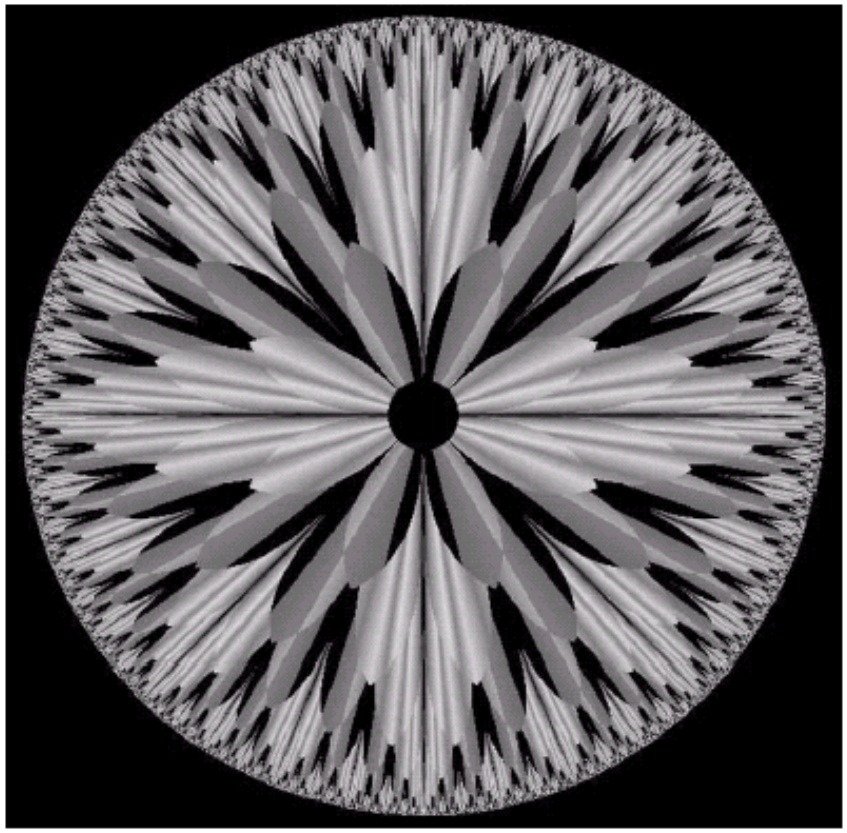


**FIGURE 3.13** An 8-bit fractal image. (A fractal is an image generated from mathematical expressions). (Courtesy of Ms. Melissa D. Binde, Swarthmore College, Swarthmore, PA.)

# Bit-plane representation

https://en.wikipedia.org/wiki/Bit_plane

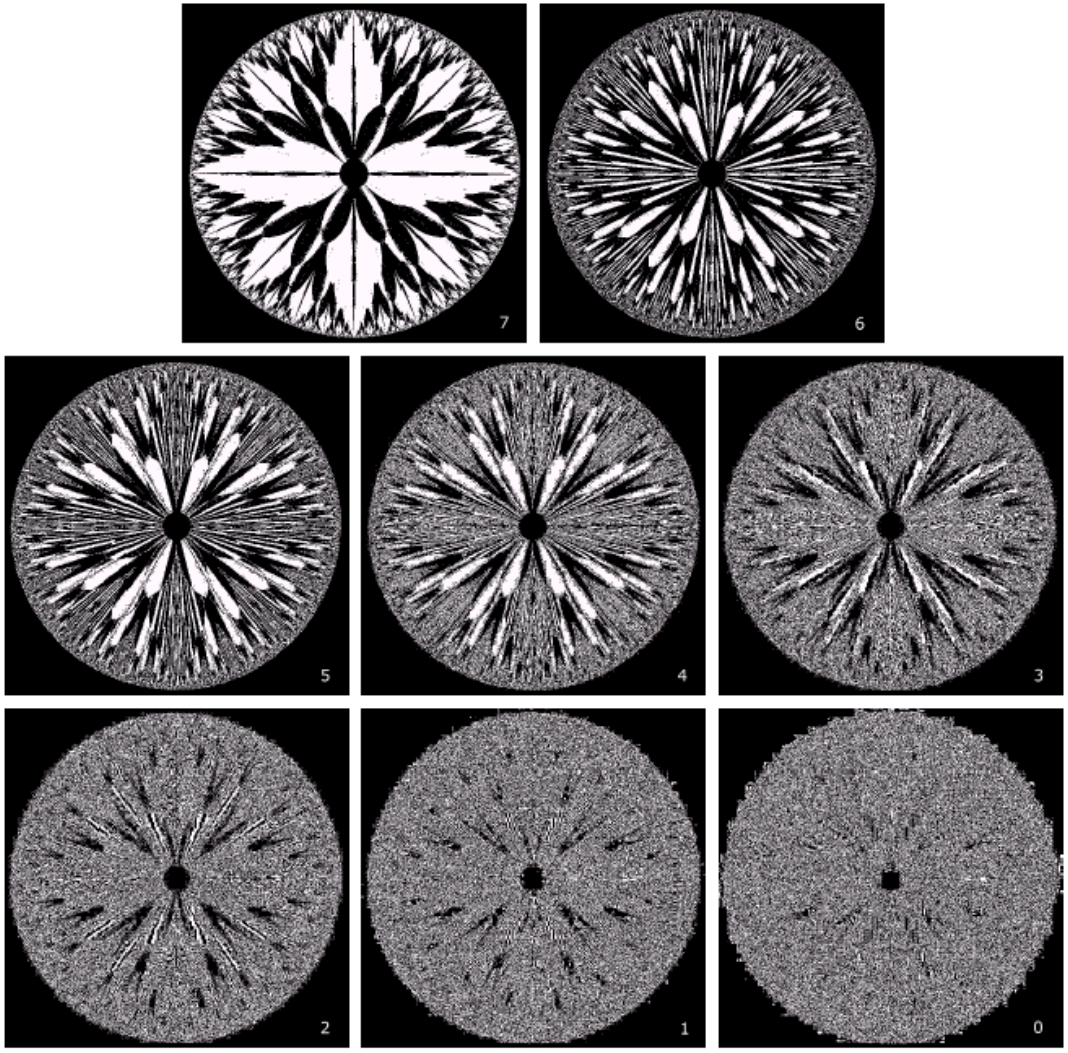



**FIGURE 3.14** The eight bit planes of the image in Fig. 3.13. The number at the bottom, right of each image identifies the bit plane.

Observation: low order bits are noisy and nearly random.

# Fidelity Criteria

1. Compression may result in loss of information, e.g., removal of psychovisually redundant data results in a loss of real or quantitative visual information (lossy compression).
2. Two criteria for assessing the compression quality.
   a. Objective fidelity criteria
   b. Subjective fidelity criteria

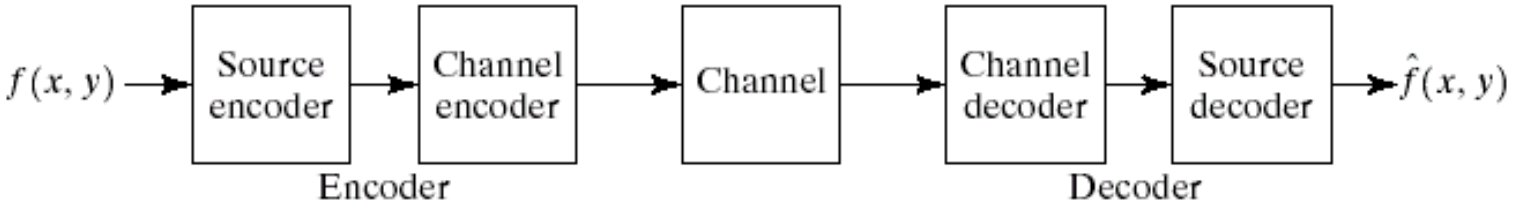



**FIGURE 8.5** A general compression system model.

# Fidelity Criteria

3. Objective fidelity criterion 1: *root-mean-square error*
4. Let $f(x,y)$ and $\hat{f}(x,y)$ be the input and approximated (after de-compression) images, respectively.
5. The root-mean-square error is defined as

$$e_{\text{rms}} = \left[ \frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} \left[ \hat{f}(x,y) - f(x,y) \right]^2 \right]^{\frac{1}{2}}$$

6. Objective fidelity criterion 2: *mean square signal-to-noise ratio*

$$SNR_{\text{ms}} = \frac{\sum_{x=0}^{M-1} \sum_{y=0}^{N-1} \hat{f}(x,y)^2}{\sum_{x=0}^{M-1} \sum_{y=0}^{N-1} \left[ \hat{f}(x,y) - f(x,y) \right]^2}$$

# Fidelity Criteria

7. Although objective fidelity criteria offer a simple and convenient mechanism for evaluating information loss, most of de-compressed images ultimately are viewed by humans.
8. Subjective fidelity criteria, based on rating, can be used to quantify the image quality after compression and de-compression.

**TABLE 8.3**
Rating scale of the Television Allocations Study Organization. (Frendendall and Behrend.)

| Value | Rating | Description |
|---|---|---|
| 1 | Excellent | An image of extremely high quality, as good as you could desire. |
| 2 | Fine | An image of high quality, providing enjoyable viewing. Interference is not objectionable. |
| 3 | Passable | An image of acceptable quality. Interference is not objectionable. |
| 4 | Marginal | An image of poor quality; you wish you could improve it. Interference is somewhat objectionable. |
| 5 | Inferior | A very poor image, but you could watch it. Objectionable interference is definitely present. |
| 6 | Unusable | An image so bad that you could not watch it. |

# Image Compression Model

# Image Compression Model

1. Compression system consists of two distinct structural blocks: an encoder and a decoder.
2. Let $f(x,y)$ and $\hat{f}(x,y)$ be the input and reconstructed/approximated (after de-compression) images, respectively.
3. If the de-compressed image is an exact replica of the input image, then the system is error free or information preserving.
4. If not, some level of distortion is present in the reconstructed image.

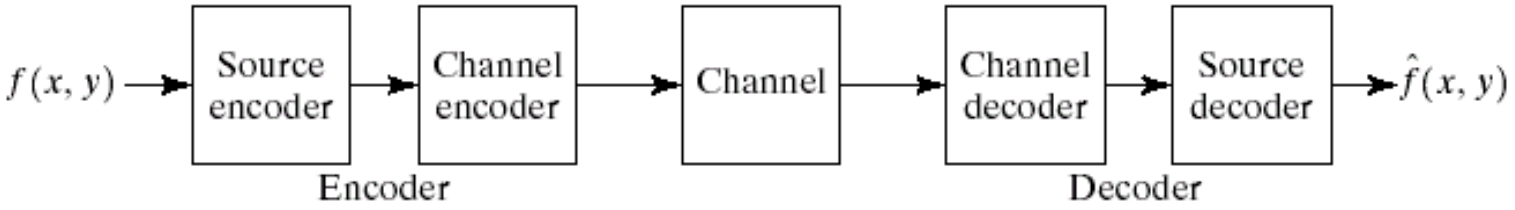



**FIGURE 8.5** A general compression system model.

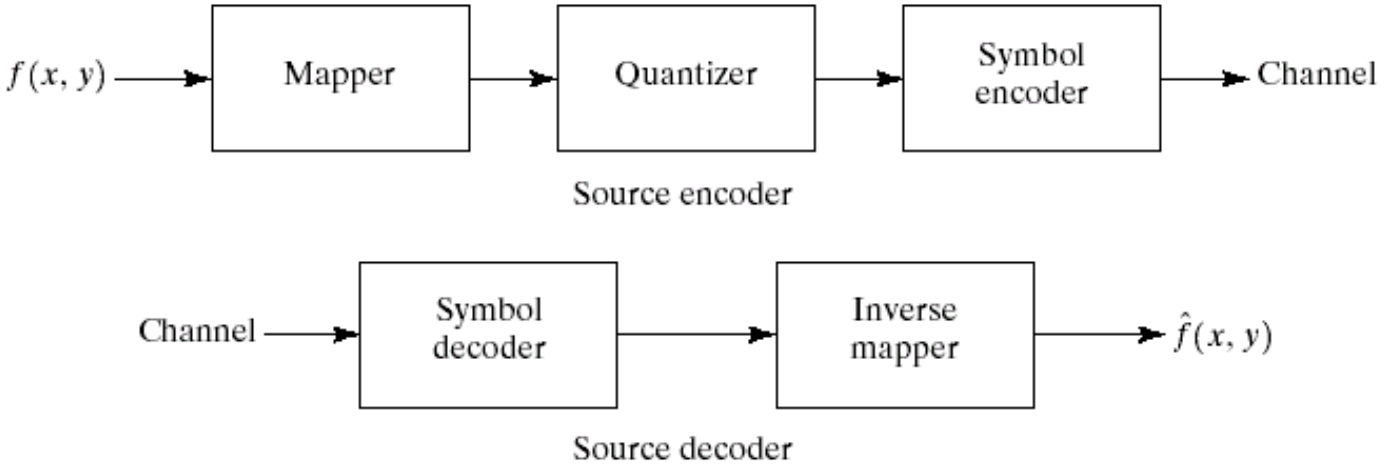
The figure is provided by Pearson Education, Digital Image Processing, Gonzalez & Woods, www.ImageProcessingPlace.com

# Image Compression Model

5. *Source encoder*: it removes redundancies, e.g., coding, interpixel, and psychovisual redundancies.
6. *Channel encoder*: it increases noise immunity of the source encoder's output. If the channel is noise free, then the channel encoder can be omitted.

# The Source Encoder and Decoder

1. The source encoder: it reduces or eliminates any coding, interpixel, or psychovisual redundancies.
2. Its quality can be assessed based on the fidelity criteria.
3. The source encoder can be modelled by a series of three independent operations: *the mapper*, *the quantizer*, and *the symbol encoder*.

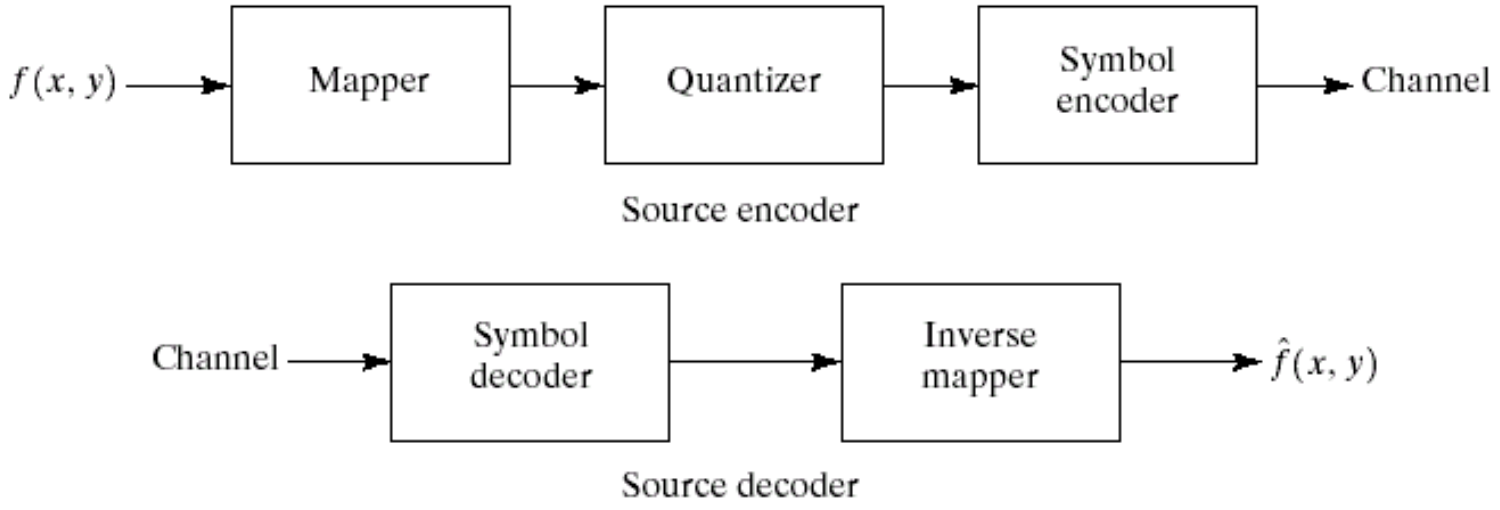



a
b

**FIGURE 8.6** (a) Source encoder and (b) source decoder model.

# The Source Encoder and Decoder

4. *The mapper*: it transforms input data into a (usually nonvisual) format designed to reduce interpixel redundancies in the input image, e.g., run-length coding, or compression based on transform coefficients.

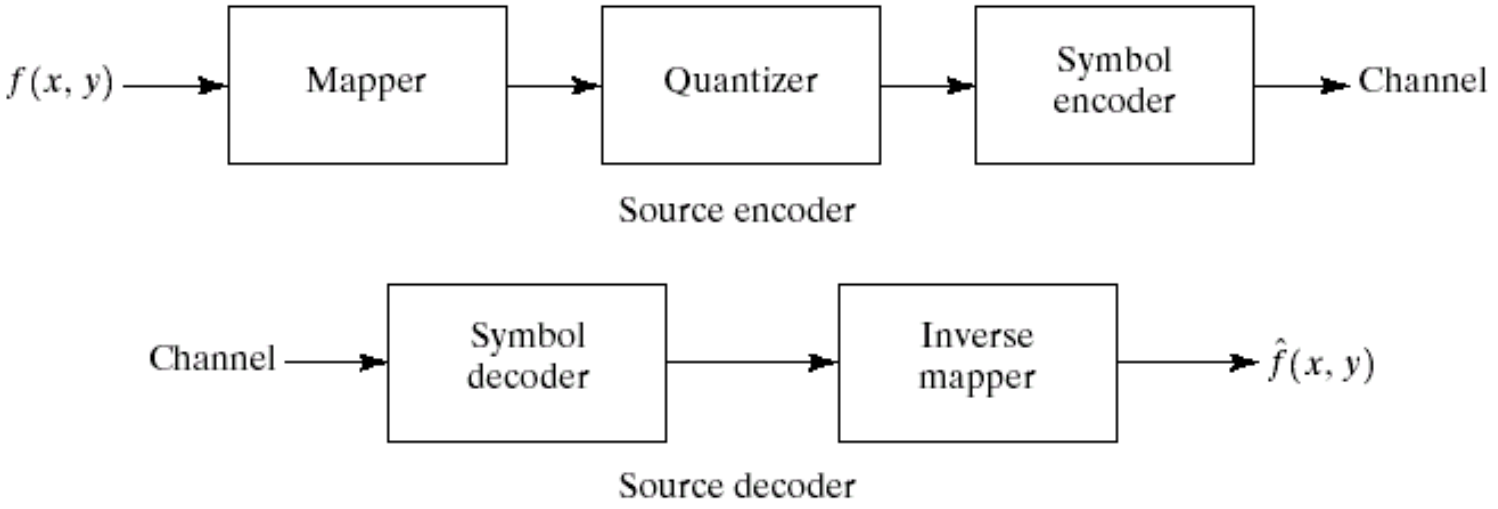



a
b

**FIGURE 8.6** (a) Source encoder and (b) source decoder model.

# The Source Encoder and Decoder

5. *The quantizer*: it reduces the accuracy of the mapper's output in accordance with some pre-established fidelity criterion, and can be used to reduce psychovisual redundancy, e.g., IGS quantization (irreversible operation).

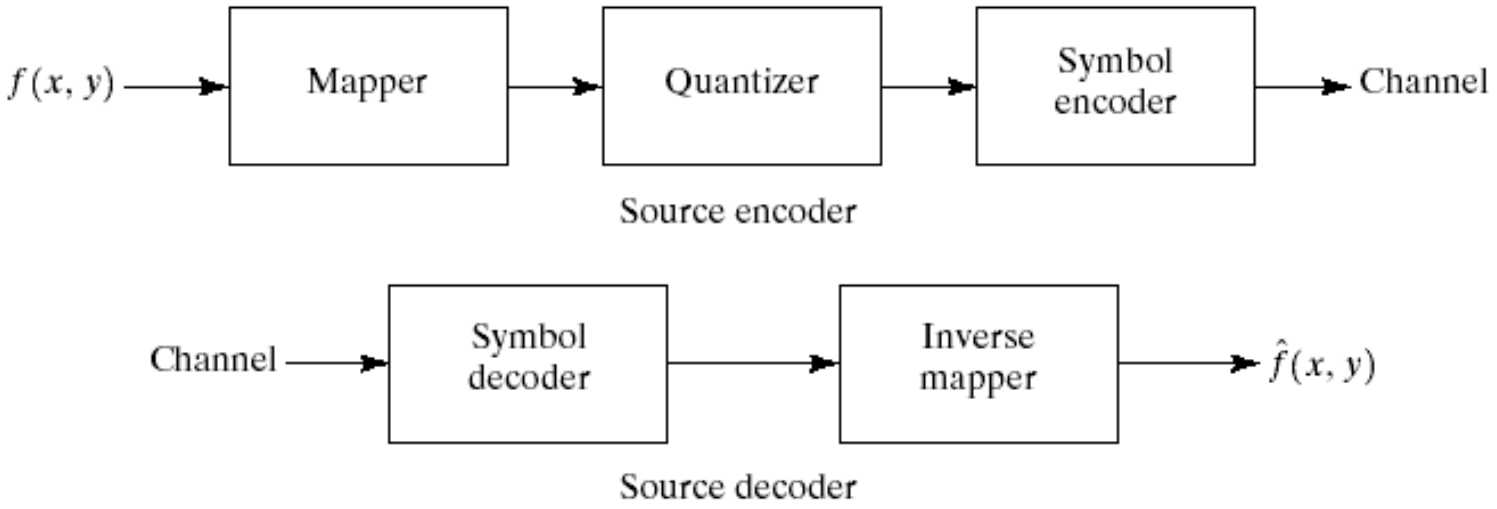



a
b

**FIGURE 8.6** (a) Source encoder and (b) source decoder model.

# The Source Encoder and Decoder

6. *The symbol coder*: it creates a fixed-length code or variable-length code to represent the quantizer output and maps the output in accordance with the code.




a
b

**FIGURE 8.6** (a) Source encoder and (b) source decoder model.

# The Source Encoder and Decoder

7. In most cases, a variable-length code is used to represent the mapped and quantized dataset.
8. The variable-length coding scheme assigns the shortest code words to the most frequently occurring output values and thus reduces coding redundancy (reversible operation).
9. For the *source decoder*, it consists of symbol decoder and an inverse mapper.
10. Since quantization results in irreversible information loss, an inverse quantiser block is not included in the general source decoder model.

# The Channel Encoder and Decoder

1. As the output of the source encoder contains little redundancy, it would be highly sensitive to transmission noise.
2. *The channel encoder*: it reduces or eliminates the impact of channel noise by inserting a controlled form of redundancy into the source encoded data.
3. Hamming showed that, if 3 bits of redundancy are added to a 4-bit word, all single-bit errors can be detected and corrected – the 7-bit Hamming (7,4) code word.

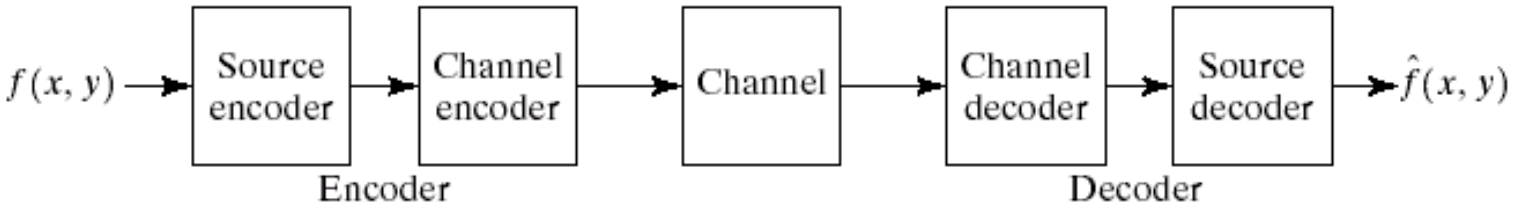



**FIGURE 8.5** A general compression system model.

# The Channel Encoder and Decoder

4. The 7-bit Hamming (7,4) code word $h_1 h_2 \ldots h_5 h_6 h_7$ with 7 bits.
5. Let $b_3 b_2 b_1 b_0$ be a 4-bit binary number with 4 data bits.
6. Each bit of the Hamming code word is defined by

$$
\begin{aligned}
h_1 &= b_3 \oplus b_2 \oplus b_0 \\
h_2 &= b_3 \oplus b_1 \oplus b_0 \\
h_3 &= b_3 \\
h_4 &= b_2 \oplus b_1 \oplus b_0 \\
h_5 &= b_2 \\
h_6 &= b_1 \\
h_7 &= b_0
\end{aligned}
$$

where $\oplus$ represents the exclusive OR operation.

# The Channel Encoder and Decoder

7. Note that bits $h_1$, $h_2$ and $h_4$ are even parity bits for the bit field $b_3b_2b_0$, $b_3b_1b_0$, and $b_2b_1b_0$, respectively.
8. A string of binary bits has even parity if the number of bits with a value of 1 is even.
9. A single-bit error is indicated by a non-zero word $c_4c_2c_1$ with 3 even parity bits, where

$$c_1 = h_1 \oplus h_3 \oplus h_5 \oplus h_7$$
$$c_2 = h_2 \oplus h_3 \oplus h_6 \oplus h_7$$
$$c_4 = h_4 \oplus h_5 \oplus h_6 \oplus h_7$$

10. If a non-zero value is found, then the decoder simply complements the cord word bit position indicated by the parity word.

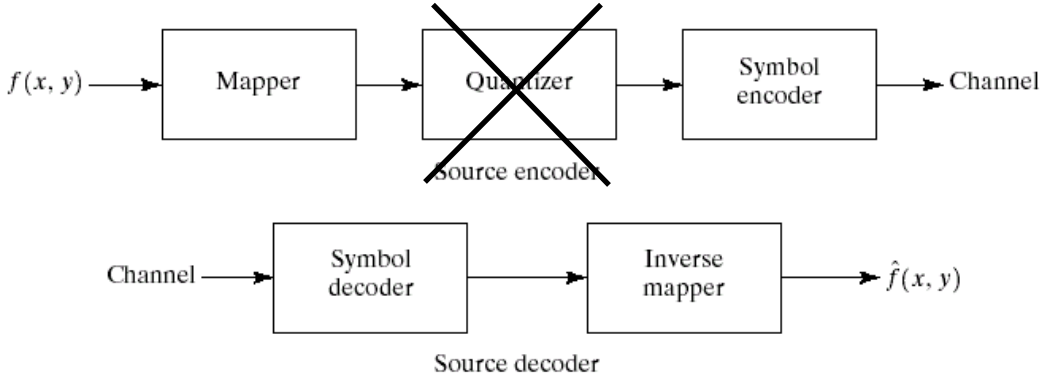
# The Channel Encoder and Decoder

11. The decoded binary value is then extracted from the corrected code word as $h_3 h_5 h_6 h_7$.
12. Example: 3 extra bits are added into the previous 4-bit IGS data example.
13. As such, the original 2:1 compression ratio is reduced to 8/7 or 1.14:1.
14. This reduction in compression ratio is the price paid for increasing noise immunity (any single bit error can be detected and corrected).

# Lossless Compression

# Error free compression

1. In many applications, error-free compression is the only acceptable means of data reduction, e.g., archival of medical or business documents, where lossy compression usually is prohibited for legal reasons.
2. Compression ratios are from 2 to 10.
3. Two general operations
   a. reduce interpixel redundancy and
   b. eliminate coding redundancy.




a
b

**FIGURE 8.6** (a) Source encoder and (b) source decoder model.

# Huffman Encoding

1. Natural binary encoding of the grey levels normally has coding redundancy, e.g., fixed-length coding.
2. Solution: to assign the shortest possible code words to the most probable grey levels.
3. It creates a series of source reductions by ordering the probabilities of the symbols and combining the lowest probability symbols into a single symbol that replaces them in the next source reduction.

| Original source | | Source reduction | | | |
|---|---|---|---|---|---|
| Symbol | Probability | 1 | 2 | 3 | 4 |
| $a_2$ | 0.4 | 0.4 | 0.4 | 0.4 | 0.6 |
| $a_6$ | 0.3 | 0.3 | 0.3 | 0.3 | 0.4 |
| $a_1$ | 0.1 | 0.1 | 0.2 | 0.3 | |
| $a_4$ | 0.1 | 0.1 | 0.1 | | |
| $a_3$ | 0.06 | 0.1 | | | |
| $a_5$ | 0.04 | | | | |

**FIGURE 8.11**
Huffman source reductions.

| Original source | | | Source reduction | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Sym. | Prob. | Code | 1 | | 2 | | 3 | | 4 | | | |
| $a_2$ | 0.4 | 1 | 0.4 | 1 | 0.4 | 1 | 0.4 | 1 | 0.6 | 0 | | |
| $a_6$ | 0.3 | 00 | 0.3 | 00 | 0.3 | 00 | 0.3 | 00 | 0.4 | 1 | | |
| $a_1$ | 0.1 | 011 | 0.1 | 011 | 0.2 | 010 | 0.3 | 01 | | | | |
| $a_4$ | 0.1 | 0100 | 0.1 | 0100 | 0.1 | 011 | | | | | | |
| $a_3$ | 0.06 | 01010 | 0.1 | 0101 | | | | | | | | |
| $a_5$ | 0.04 | 01011 | | | | | | | | | | |

**FIGURE 8.12**
Huffman code assignment procedure.

# Huffman Encoding

5. The second step in Huffman's procedure is to encode each reduced source, starting with the smallest source and working back to the original source.
6. The average length of new codes

$$L_{avg} = (0.4)(1)+(0.3)(2)+(0.1)(3)+(0.1)(4)+(0.06)(5)+(0.04)(5)$$

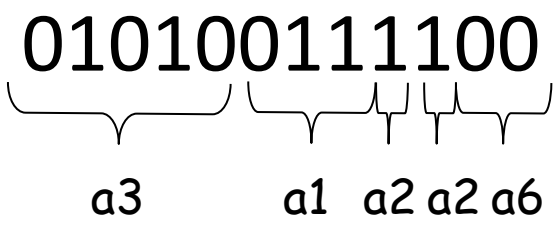
$$= 2.2 \text{ bits}$$

Compression ratio and relative data redundancy?

# Huffman Decoding

7. It is uniquely encodable because any string of code symbols can be encoded in only one way.

- consider




Symbol Table

| | | |
|---|---|---|
| a2 | 0.4 | **1** |
| a6 | 0.3 | **00** |
| a1 | 0.1 | **011** |
| a4 | 0.1 | **0100** |
| a3 | 0.06 | **01010** |
| a5 | 0.04 | **01011** |

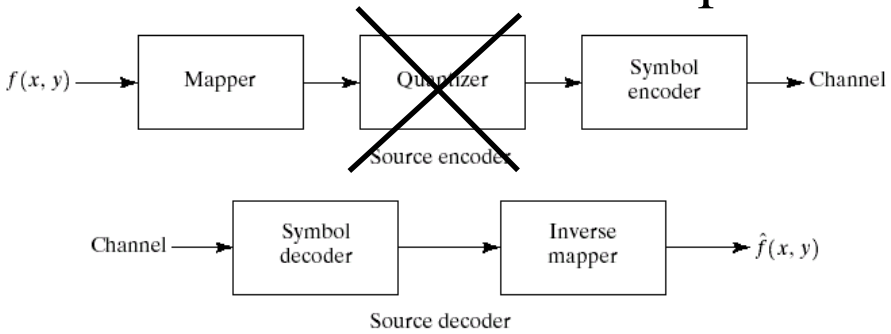
# Huffman Coding

8. It is a very powerful coding scheme.
9. Produces the optimal codes
    a. given an input symbol set
    b. and its known distribution

10. Need to store symbol table along with compressed data!

# Lossless predictive coding

1. Lossless predictive coding is designed based on reducing the interpixel redundancies of closely spaced pixels by coding only the new information in each pixel.
2. *New information* = the difference between the actual and predicted values of that pixel.
3. The system consists of an encoder and a decoder.
4. *Encoder*:
   a. It consists of a predictor, which generates the predicted value based on some of the past values.




a
b

**FIGURE 8.6** (a) Source encoder and (b) source decoder model.

# Lossless predictive coding

$$f_n' = \text{round}\left[\sum_{i=1}^{m} \alpha_i f_{n-i}\right]$$

where $f_n'$ = the predicted image (intensity) value

$n$ = image index, $n = 1, \ldots, N$

$\alpha_i$ = weights

$f_n$ = image (intensity) value at index $n$

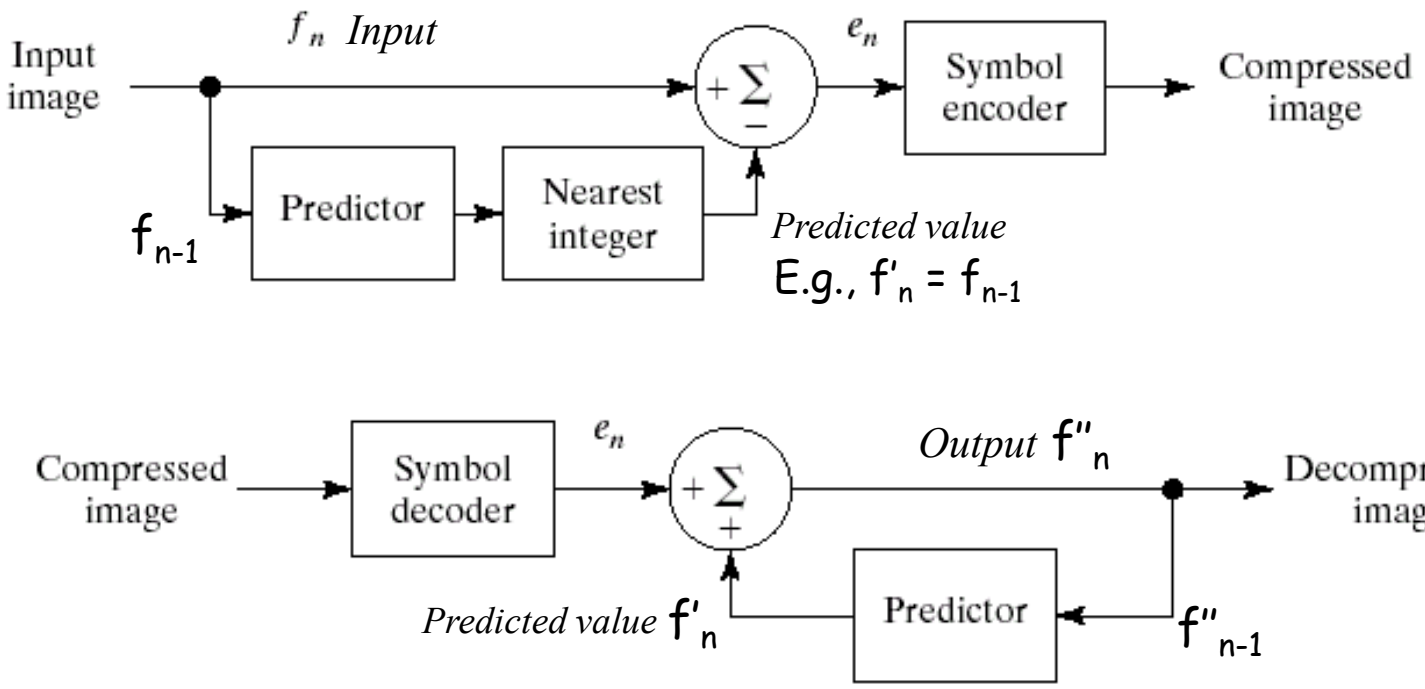
Another representation

$$f_n' = \text{round}\left[\sum_{i=1}^{m} \alpha_i f\left(x, y-i\right)\right]$$

# Lossless predictive coding

4. *Encoder*:
   b. The prediction error between the input and predicted value is computed and encoded for transmission.

$$e_n = f_n - f_n'$$




a
b

**FIGURE 8.19** A lossless predictive coding model: (a) encoder; (b) decoder.

# Lossless predictive coding

5. *Decoder*:
   a. It consists of a predictor, which is *exactly the same* as the predictor in the encoding procedure.
   b. The image value is generated by adding the transmitted prediction error value $e_n$ and the predicted value.
6. Since the first *m* pixels cannot be evaluated, these pixels may need to be encoded by using other compression methods. It can be viewed as the overhead of this compressing scheme.

# Lossless predictive coding (Example)

1. *Differential coding or previous pixel coding*
2. How does it work? The predicted value is computed solely from the immediate adjacent pixel value, and alpha is set to 1. As such, the prediction error represents the difference between the input $f(x, y)$ and predicted value $f' = f(x, y-1)$.
3. That is

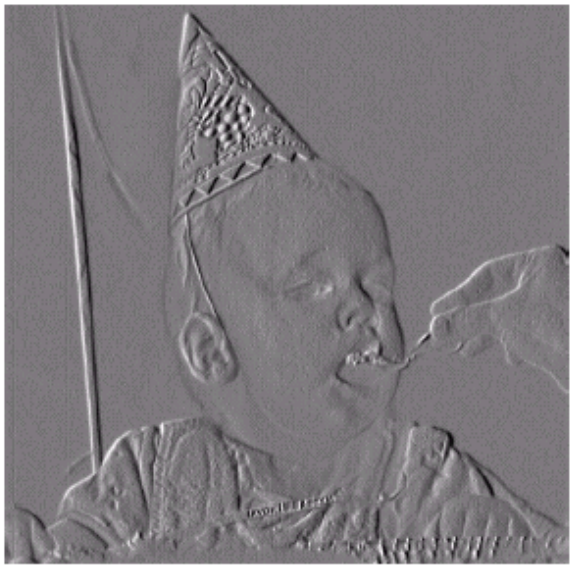
$$\alpha = 1$$
$$m = 1$$
$$e_n = f(x, y) - f(x, y-1)$$
$$\textit{current pixel} - \textit{predicted pixel}$$

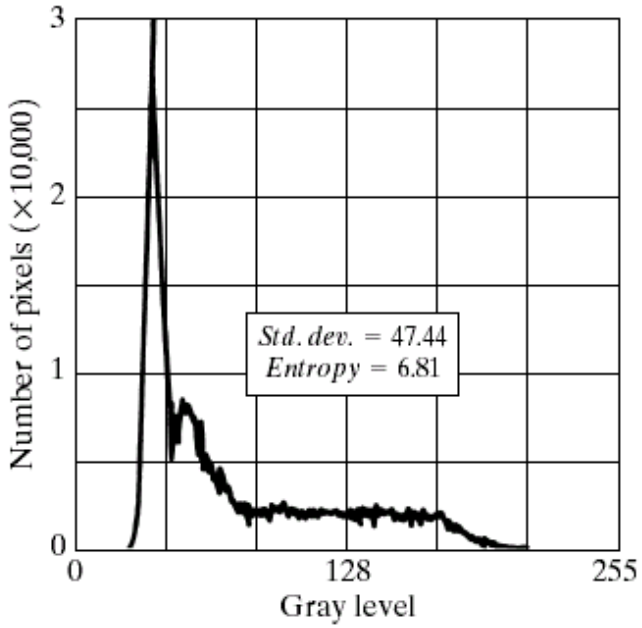
We assume 1D line compression in this example.

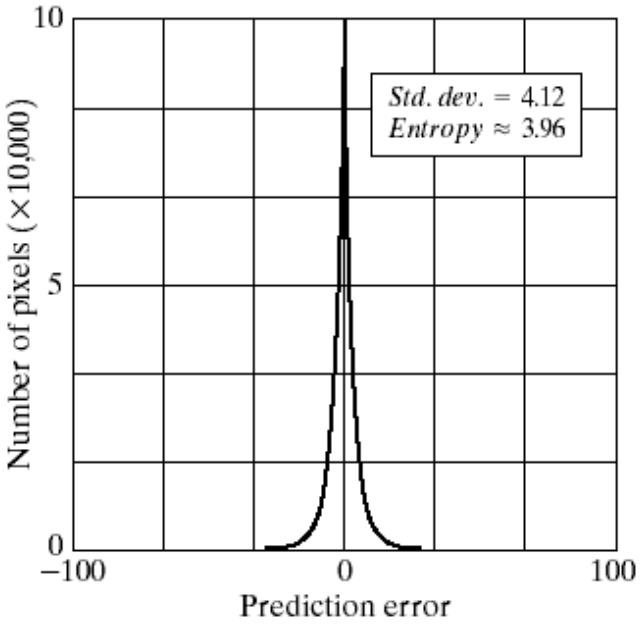
*Original, input image*

a
b c

**FIGURE 8.20**
(a) The prediction error image resulting from Eq. (8.4-9).
(b) Gray-level histogram of the original image.
(c) Histogram of the prediction error.





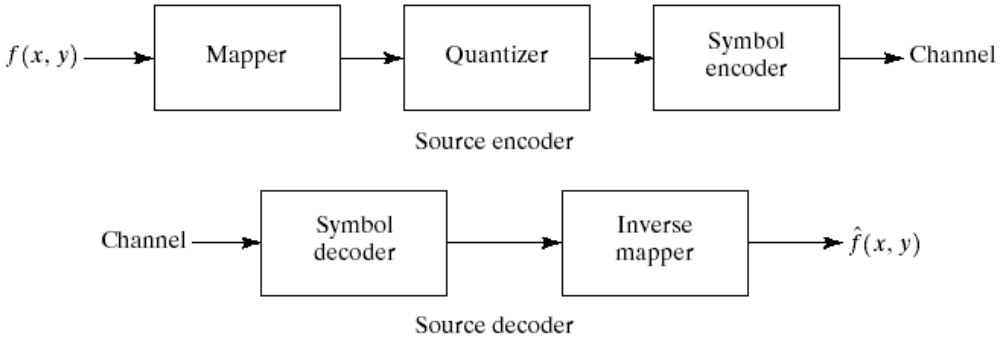
# Lossless predictive coding (Example)

4. Uncompressed image: mean = 128, SD = 47.44
5. Compressed image (prediction error): mean = 0, SD = 4.12
6. The variance of the prediction errors is much smaller than the variance of the grey levels in the original image.
7. Average lengths for the original image and the prediction error image are 8 bits and 3.96 bits, respectively, e.g., variable length coding or Huffman coding can be used.
8. The compression ratio is about 2:1.

# Lossy Compression

# Lossy Compression

- Compromise accuracy
  - That is, we will allow for “error” and distortion
- In exchange for increased compression
  - If the resulting distortion can be tolerated, then we can gain substantial compression
- Exploit psycho-visual redundancy
  - Our eye is less sensitive to high-frequencies
  - Can we “throw” away some details and make the processed image look similar to the original image?




a
b

**FIGURE 8.6** (a) Source encoder and (b) source decoder model.

The figure is provided by Pearson Education, Digital Image Processing, Gonzalez & Woods, www.ImageProcessingPlace.com

# Lossy vs. Lossless

- Lossy Schemes
  - Image is still recognisable
    - 30:1 Compression ratio
  - Image is “virtually” indistinguishable from original
    - 20:1 - 10:1 Compression ratio
- Lossless
  - Rarely get more than 3:1 reduction

# Lossy Schemes

- Principal difference from lossless

- Introduction of “quantization”
  - Quantization maps values to a limited range
  - The more the quantization
    - More Compression
    - (and more distortion)

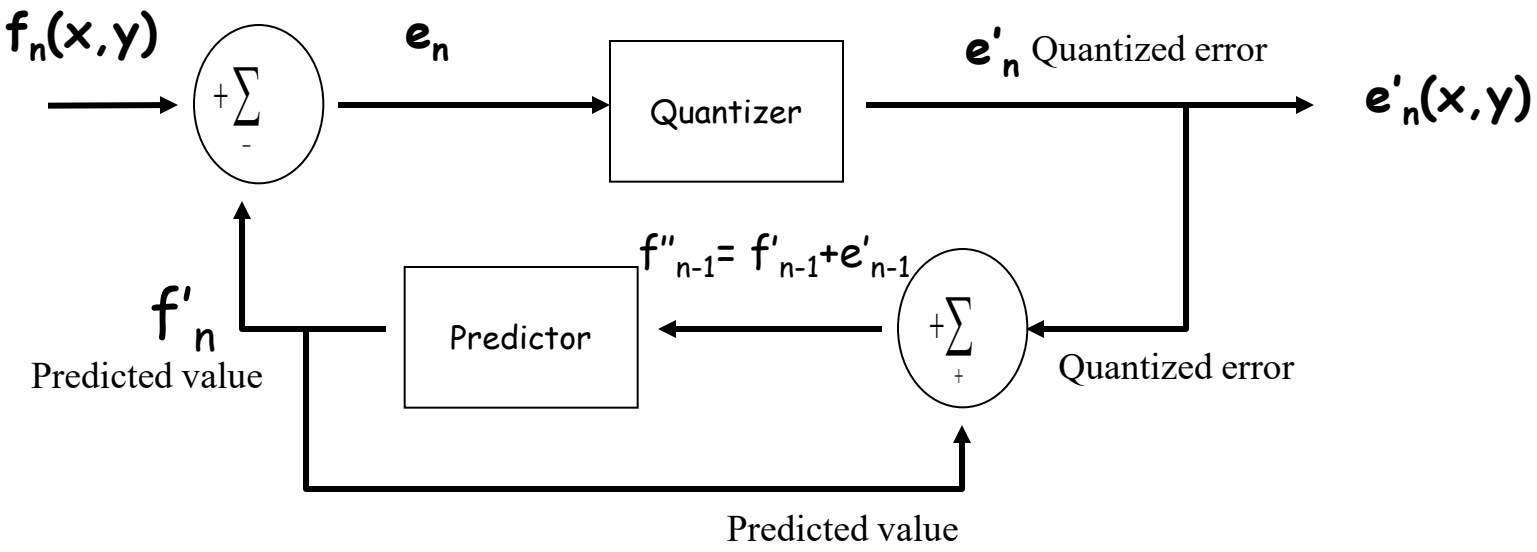
# Lossy Predictive Coding

- Predictive Coding Scheme
  - Based on a lossless predictive coding scheme
  - Recall that lossless prediction was:
    - $e_n = f_n - f'_n$
      - $f_n$ is the current pixel
      - $f'_n$ is the predicted pixel
      - $e_n$ is the error

# Lossy Predictive Coding

- Errors will be quantized to a limited range
  - $e_n = f_n - f'_n$
  - $e'_n = e_n / q$ (quantization)
  - $f''_n = f'_n + e'_n$ (resulting/reconstructed pixel with error)
    - $f_n$ is the current pixel
    - $f'_n$ is the predicted pixel
    - $e'_n$ is the quantized error
    - $f''_n$ is the resulting (reconstructed) pixel
      - note, we will need to use $f''_n$ in the predictor

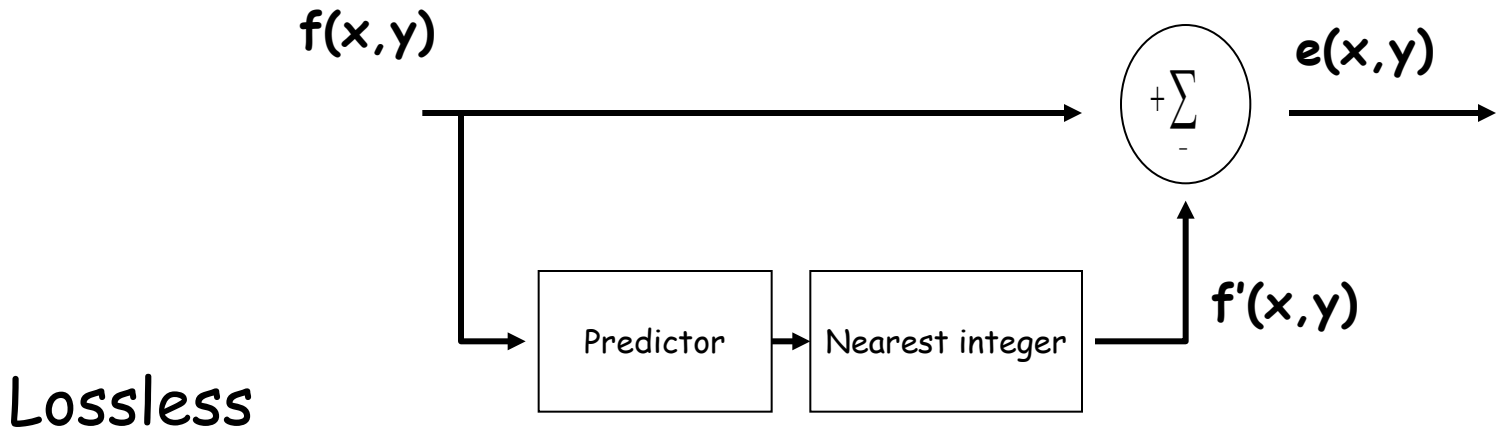
# Lossy Predictive Coding Model

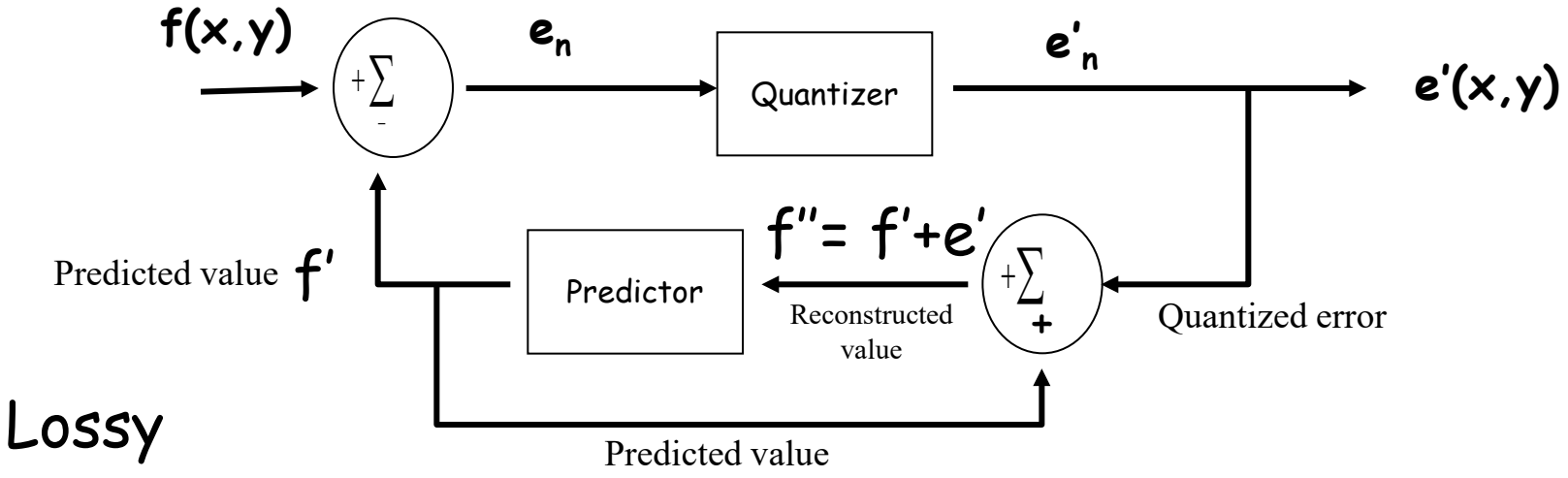



- Error is quantized
- Predictor uses lossy pixels ($f''_{n-1}$) to predict $f'_n$

The figure is provided by Pearson Education, Digital Image Processing, Gonzalez & Woods, www.ImageProcessingPlace.com

# Lossy/Lossless Comparison



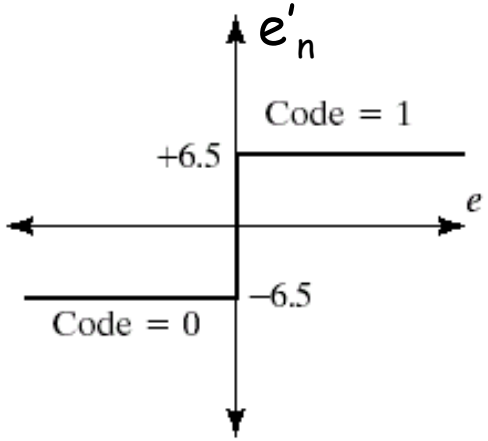

The figure is provided by Pearson Education, Digital Image Processing, Gonzalez & Woods, www.ImageProcessingPlace.com

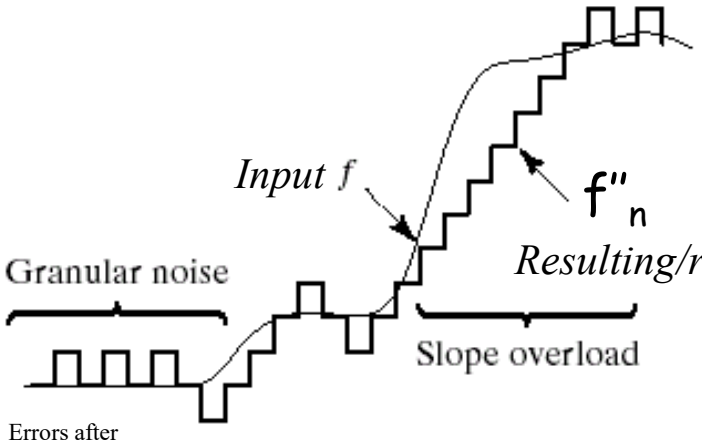
# Delta Modulation

- Simple and well-known technique
- Predictor = $f'_n = f''_{n-1}$
- $e'_n = \begin{cases} +\gamma & e_n > 0 \\ -\gamma & \text{otherwise} \end{cases}$
- where $\gamma$ is a constant

# Example 1-D scanline

$\gamma = m$




a b c

**FIGURE 8.22** An example of delta modulation.

| Input | | Encoder: Predicted values | | Encoder: Reconstructed values and inputs to the predictor | | Decoder | | Error |
|---|---|---|---|---|---|---|---|---|
| $n$ | $f_n$ | $f'_n$ | $e_n$ | $e'_n$ | $f''_n$ | $f'_n$ | $f''_n$ | $f_n - f''_n$ |
| 0 | 14 | — | — | — | 14.0 | — | 14.0 | 0.0 |
| 1 | 15 | 14.0 | 1.0 | 6.5 | 20.5 | 14.0 | 20.5 | –5.5 |
| 2 | 14 | 20.5 | –6.5 | –6.5 | 14.0 | 20.5 | 14.0 | 0.0 |
| 3 | 15 | 14.0 | 1.0 | 6.5 | 20.5 | 14.0 | 20.5 | –5.5 |
| . | . | . | . | . | . | . | . | . |
| . | . | . | . | . | . | . | . | . |
| 14 | 29 | 20.5 | 8.5 | 6.5 | 27.0 | 20.5 | 27.0 | 2.0 |
| 15 | 37 | 27.0 | 10.0 | 6.5 | 33.5 | 27.0 | 33.5 | 3.5 |
| 16 | 47 | 33.5 | 13.5 | 6.5 | 40.0 | 33.5 | 40.0 | 7.0 |
| 17 | 62 | 40.0 | 22.0 | 6.5 | 46.5 | 40.0 | 46.5 | 15.5 |
| 18 | 75 | 46.5 | 28.5 | 6.5 | 53.0 | 46.5 | 53.0 | 22.0 |
| 19 | 77 | 53.0 | 24.0 | 6.5 | 59.6 | 53.0 | 59.6 | 17.5 |
| . | . | . | . | . | . | . | . | . |
| . | . | . | . | . | . | . | . | . |

= data transmitted through the channel

# Example: 2D Image

Original

$\gamma = 6.5$ roughly 8:1 Compression

# Predictive vs. Transform coding

- Predictive coding
  - In predictive coding, information already sent or available is used to predict future values, and the difference is encoded. Since this is done in the image or spatial domain, it is relatively simple to implement and is readily adapted to local image characteristics.

- Transform coding
  - Transform coding, on the other hand, first transforms the image from its spatial domain representation to a different type of representation using some well-known transform and then encodes the transformed values (coefficients). This method provides greater data compression than predictive methods, though at the expense of greater computational cost.

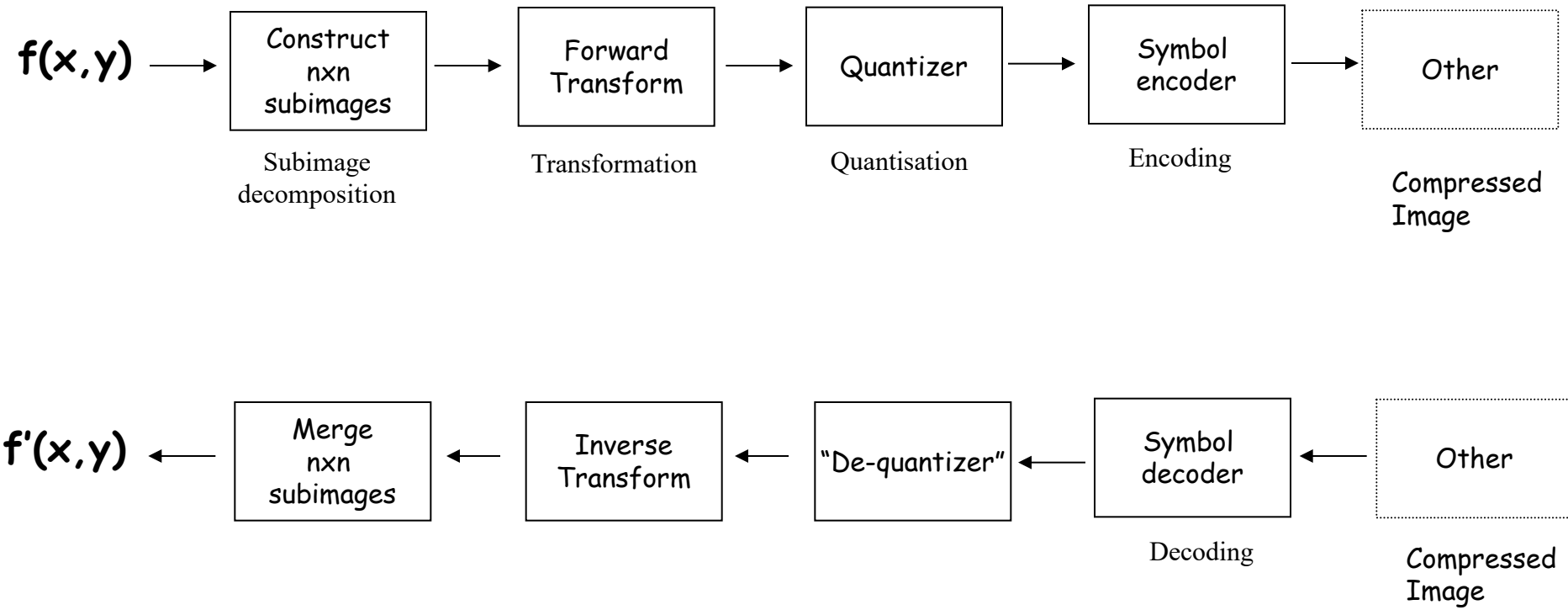
Image Compression - from DCT to Wavelets: A Review, by Subhasis Saha, Crossroads 6.3 - Spring 2000

# Transform Coding

- Predictive coding operates directly on pixels

- Transform coding
  - Uses a reversible, linear transform (Fourier Transform (FT), for example)
  - Maps the pixels to new coefficients (e.g., in FT, maps spatial information to frequency information)
  - Quantizes the coefficients
  - Compresses these further (Run Length Encoding, Variable Length Coding)

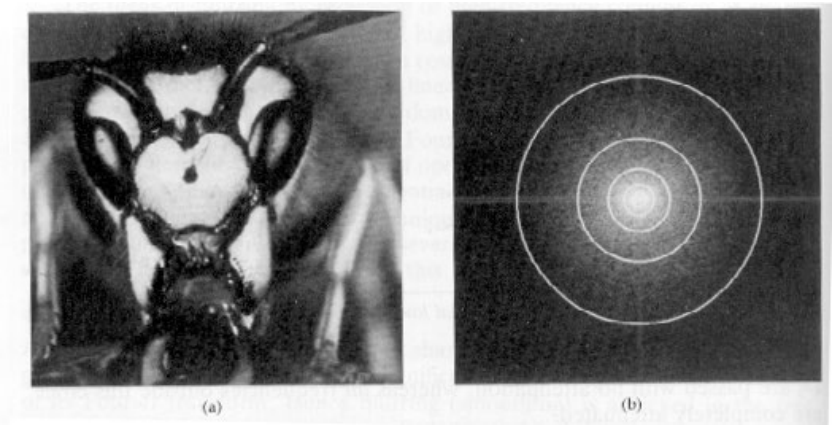
# Typical Transform Coding Scheme



# Ideas behind Transform Coding

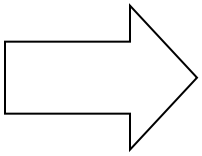
- Transform pixels to a new “space”.
- The new space provides a more compact representation of the information, or packs as much information as possible into a smaller number of transform coefficients.
  - Signal Power “Packing”.
- As such, compression is achieved during the quantization of the transformed coefficients.

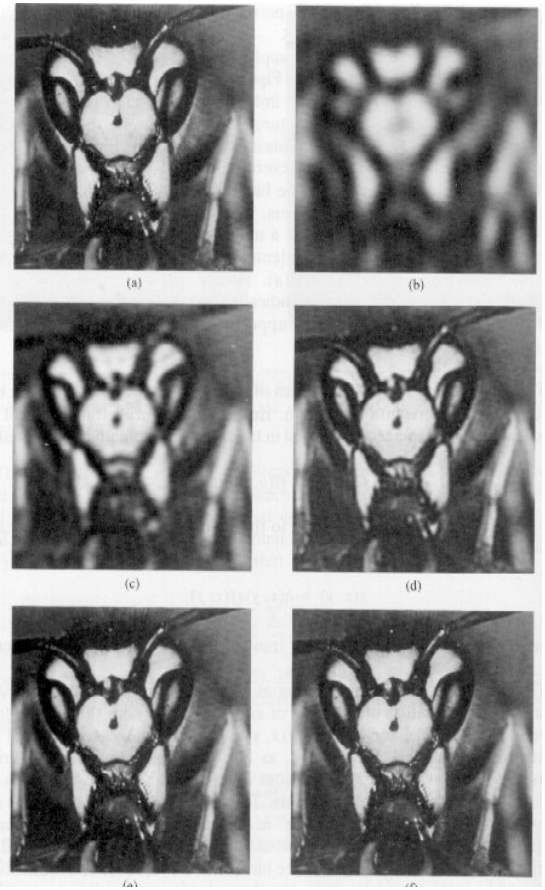
# Remember this example




Original

2D Fourier Coefficients F(u,v)






*Observation*: A significant number of the coefficients have small magnitudes and can be coarsely quantized (or discarded entirely) with little image distortion.

# Transform Coding

- Discrete Cosine Transform (DCT)
  - Like FT
  - But no imaginary component
  - DCT shown to have better power “packing” abilities over DFT

# Discrete Cosine Transform (Forward DCT)

$$C(u,v) = \alpha(u)\alpha(v)\sum_{x=0}^{N-1}\sum_{y=0}^{N-1} f(x,y)\cos\left[\frac{(2x+1)u\pi}{2N}\right]\cos\left[\frac{(2y+1)v\pi}{2N}\right]$$

$$\alpha(i) = \begin{cases} \sqrt{\frac{1}{N}} & \text{for i=0} \\ \sqrt{\frac{2}{N}} & \text{for i=1, . . ., N-1} \end{cases}$$

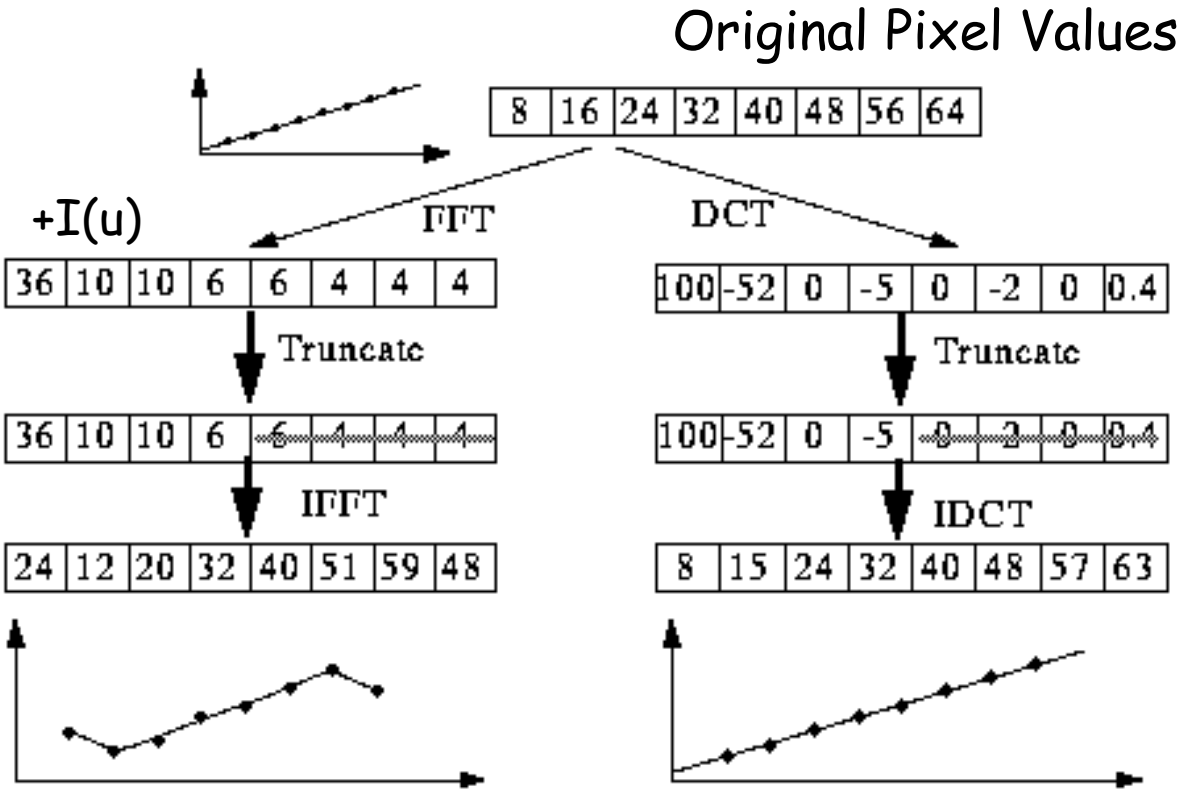
# Discrete Cosine Transform (Inverse DCT)

$$f(x,y) = \sum_{u=0}^{N-1}\sum_{v=0}^{N-1} \alpha(u)\alpha(v)C(u,v)\cos\left[\frac{(2x+1)u\pi}{2N}\right]\cos\left[\frac{(2y+1)v\pi}{2N}\right]$$

$$\alpha(i) = \begin{cases} \sqrt{\frac{1}{N}} & \text{for i=0} \\ \sqrt{\frac{2}{N}} & \text{for i=1, . . ., N-1} \end{cases}$$

# DCT vs. FFT

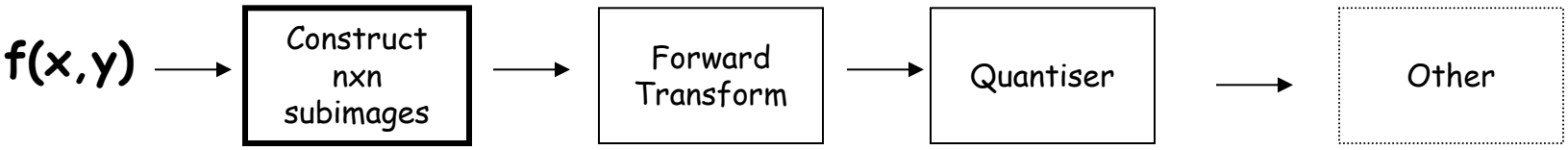
## Why DCT and not FFT?




Throw away half of the real array (not the imaginary array).

DCT behaves better under quantization. . . (and no complex math)

# Subimage size (n=?)




- Power of 2
  - Allows "fast O(*nlogn*)" algorithms to be applied
  - For example, a NxN image can be partitioned into $(N/n)^2$ subimages. Each nxn subimage can be transformed using DFT or DCT.
- As the size *n* increases
  - Level of achievable compression increases
  - Level of computational complexity increases
- Empirical testing showed
  - either n=8 or n=16 gives the best overall performance
  - testing performed back in the early 90s
  - today's computing power can probably handle larger *n*

# 2D 4x4 DCT Basis

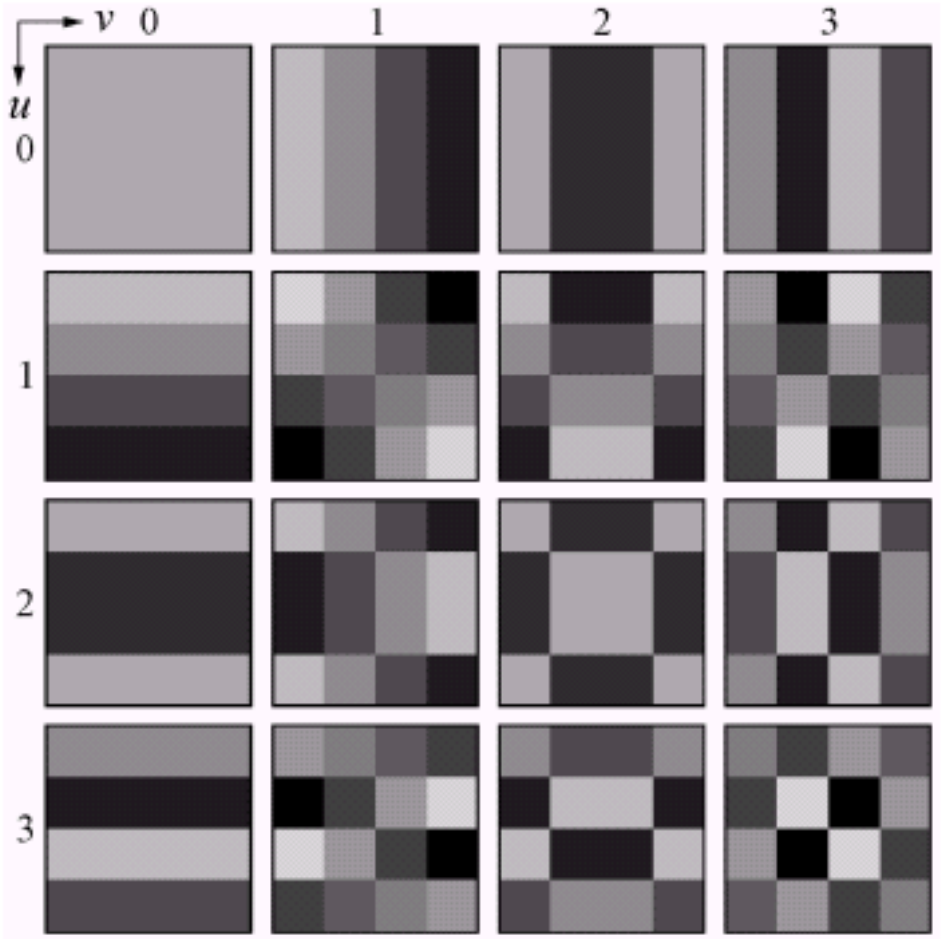
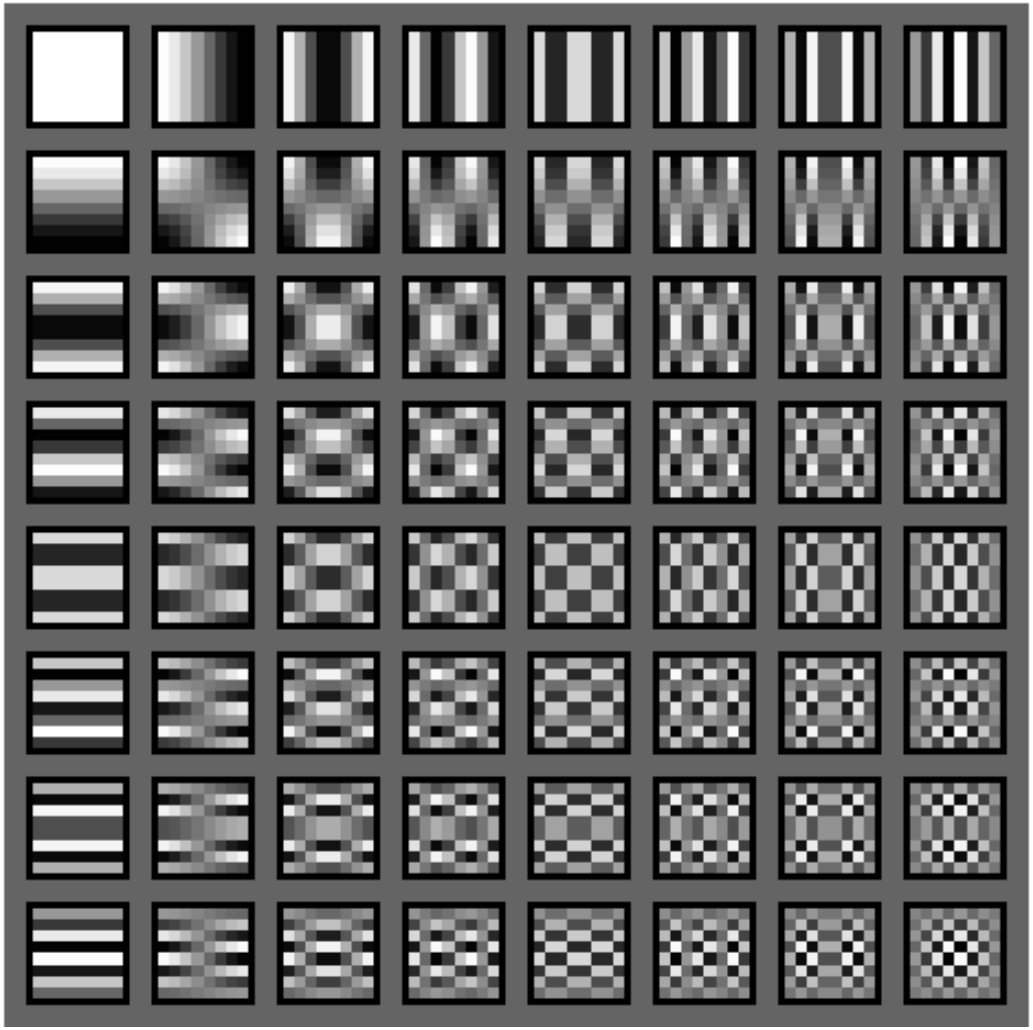



**FIGURE 8.30** Discrete-cosine basis functions for $N = 4$. The origin of each block is at its top left.

# 2D 8x8 DCT Basis

1. 2D DCT using 8x8 block.
2. Each transformed block is a linear combination of these bases. Weights are given by the coefficients.
3. One coefficient for each basis.



The figure is provided by Pearson Education, Digital Image Processing, Gonzalez & Woods, www.ImageProcessingPlace.com

# Comparisons between DFT and DCT

1. A 512x512 image, as shown below, is approximated using 8x8 subimage blocks and different transforms (DFT and DCT).
2. For each block, 50% of the resulting coefficients were truncated based on the magnitudes, i.e., 32 out of 64 coefficients. The remaining truncated coefficients were used to perform inverse transforms.

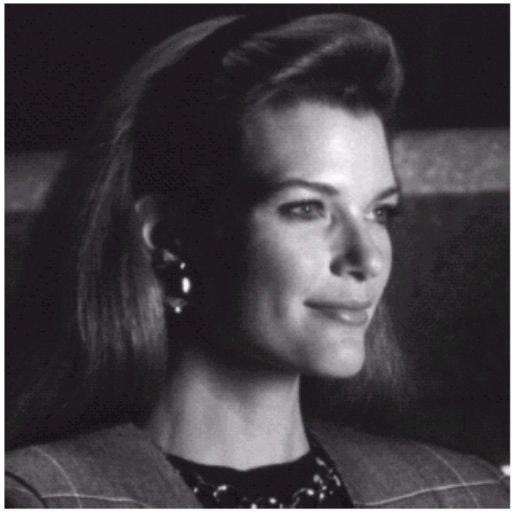
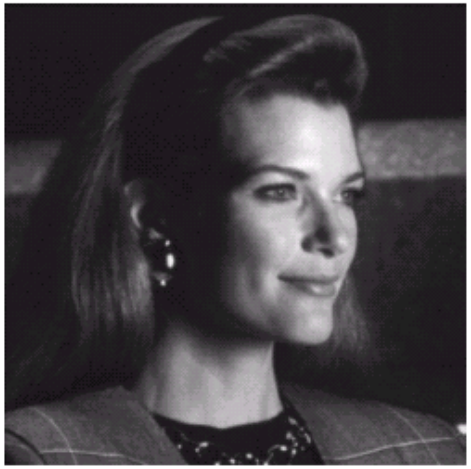


**FIGURE 8.23** A 512 × 512 8-bit monochrome image.

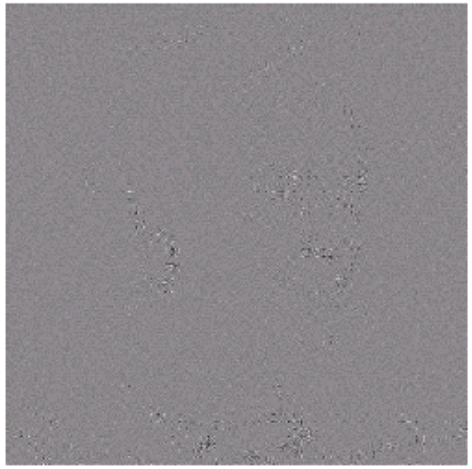
The figure is provided by Pearson Education, Digital Image Processing, Gonzalez & Woods, www.ImageProcessingPlace.com

# Comparisons between DFT and DCT

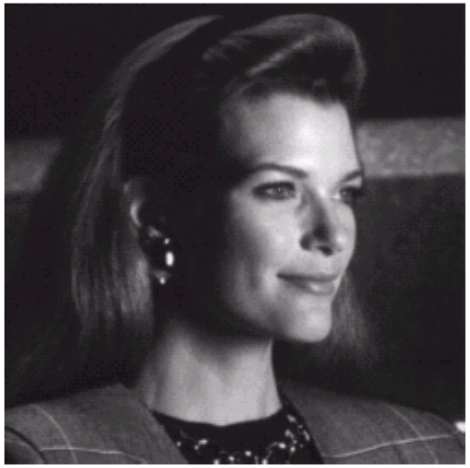
3. Based on the error images, the 32 discarded coefficients had little visual impact on reconstructed image quality.
4. The actual root mean square errors were 1.28 for DFT and 0.68 for DCT.



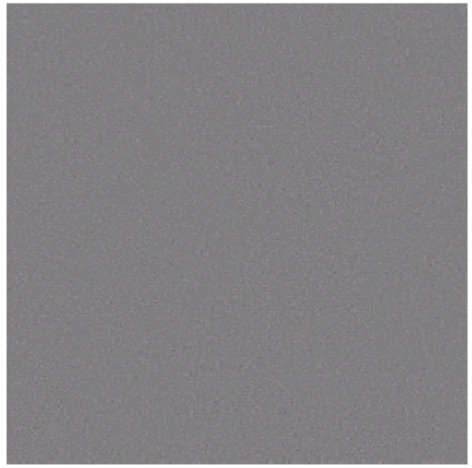
Reconstructed image based on DFT



Error image



Reconstructed image based on DCT



Error image

The figure is provided by Pearson Education, Digital Image Processing, Gonzalez & Woods, www.ImageProcessingPlace.com

# Example of compression with DCT

- matlab
  - dctdemo

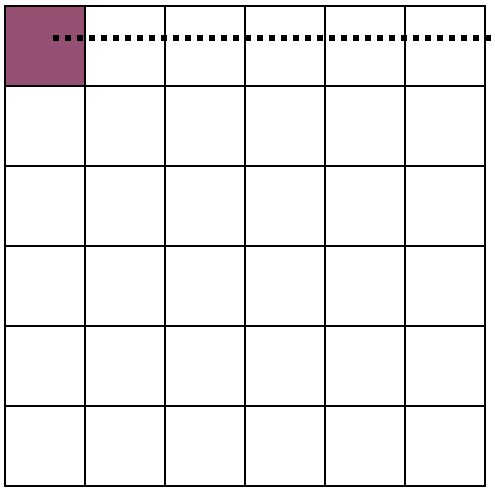
- divide image into 8x8 blocks
  - quantization in this example means dropping coefficients
  - coefficients are dropped based on their magnitude |C(u,v)|

# JPEG Compression

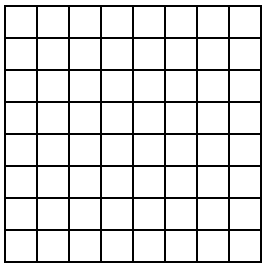
- JPEG
  - Joint Photographic Expert Group
- International Organization for Standards (ISO)
  - 1988: ISO got together a group of experts to develop a good image compression scheme
    - The scheme is geared towards photographs of natural imagery
    - Color and monochrome
    - Easy to use
  - Through empirical testing, the following scheme proved to be the best
  - Standardized in August 1990

# JPEG Encoding Scheme

f(x,y) is divided into 8x8 blocks



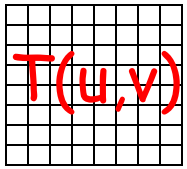
f(x,y) - 128
(normalize between –128 to 127)



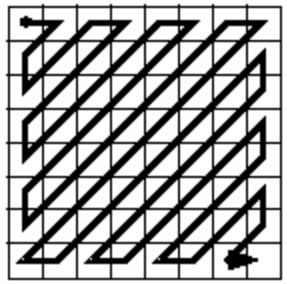
FDCT
(Forward DCT)
on each block

Quantize DCT coefficients
via
Quantization "Table"

$C'(u,v) = \text{round}(C(u,v)/T(u,v))$




2D



1D


"Zig-zag" Order Coefficients

The DC coefficient is the difference coded relative to the DC coefficient of the previous subimage.

The nonzero AC coefficients are coded using a run-length code

Huffman Encode

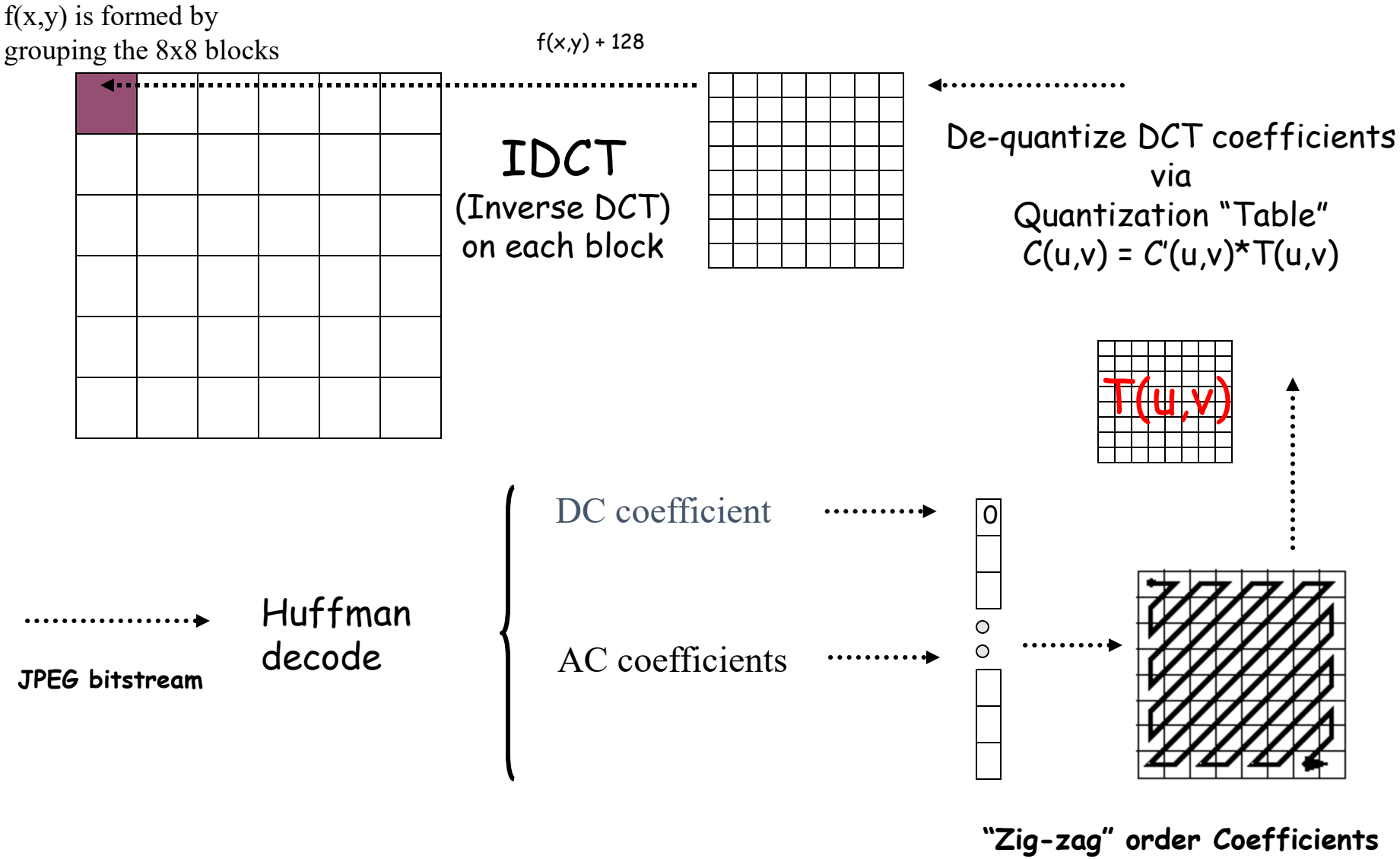
**JPEG bitstream**

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 5 | 6 | 14 | 15 | 27 | 28 |
| 2 | 4 | 7 | 13 | 16 | 26 | 29 | 42 |
| 3 | 8 | 12 | 17 | 25 | 30 | 41 | 43 |
| 9 | 11 | 18 | 24 | 31 | 40 | 44 | 53 |
| 10 | 19 | 23 | 32 | 39 | 45 | 52 | 54 |
| 20 | 22 | 33 | 38 | 46 | 51 | 55 | 60 |
| 21 | 34 | 37 | 47 | 50 | 56 | 59 | 61 |
| 35 | 36 | 48 | 49 | 57 | 58 | 62 | 63 |

# "Zig-zag" Order Coefficients

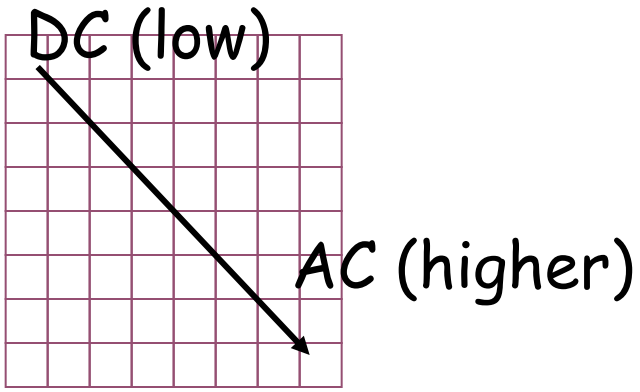
This ordering helps to place low-frequency non-zero coefficients before high-frequency coefficients.

# JPEG Decoding Scheme



# Example Quantization Table

$$T(u,v) = \begin{pmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{pmatrix}$$




Usage:

$C(u,v)$ is DCT of $f(x,y)$ 8x8 block

$C'(u,v) = C(u,v) ./ T(u,v)$

Element-wise division

Note: non-uniform quantization

# Example

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 89 | 90 | 95 | 96 | 98 | 105 | 108 | 90 |
| 80 | 91 | 96 | 94 | 90 | 110 | 102 | 110 |
| 74 | 78 | 90 | 80 | 90 | 130 | 103 | 100 |
| 80 | 20 | 19 | 80 | 91 | 101 | 120 | 130 |
| 77 | 12 | 20 | 90 | 21 | 203 | 102 | 180 |
| 65 | 20 | 35 | 48 | 80 | 103 | 104 | 122 |
| 70 | 29 | 13 | 10 | 99 | 110 | 102 | 180 |
| 88 | 19 | 23 | 19 | 90 | 120 | 120 | 130 |

8x8 block of pixels
(one subimage)

-128 →

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| -39 | -38 | -33 | -32 | -30 | -23 | -20 | -38 |
| -48 | -37 | -32 | -34 | -38 | -18 | -26 | -18 |
| -54 | -50 | -38 | -48 | -38 | 2 | -25 | -28 |
| -48 | -108 | -109 | -48 | -37 | -27 | -8 | 2 |
| -51 | -116 | -108 | -38 | -107 | 75 | -26 | 52 |
| -63 | -108 | -93 | -80 | -48 | -25 | -24 | -6 |
| -58 | -99 | -115 | -118 | -29 | -18 | -26 | 52 |
| -40 | -109 | -105 | -109 | -38 | -8 | -8 | 2 |

↓ DCT

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| -9 | -21 | -22 | -11 | -5 | 0 | -1 | 0 |
| 1 | 6 | 6 | 4 | 0 | 0 | 0 | -1 |
| 1 | 3 | 2 | -1 | 1 | -1 | 0 | -1 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | -1 | -1 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| -141.7749 | -235.1130 | -223.7993 | -179.2516 | -129.0470 | -14.8492 | -57.6292 | 6.3640 |
| 7.4404 | 77.4853 | 84.9949 | 80.5956 | 9.7875 | -9.8053 | -4.4067 | -59.7057 |
| 11.3436 | 39.7789 | 33.4318 | -29.9989 | 38.7429 | -38.9807 | 2.1976 | -28.5633 |
| -5.8066 | -7.1957 | -4.8572 | 10.9031 | -20.1451 | 8.8575 | -9.4986 | 14.8767 |
| 15.9099 | -27.2236 | -27.2236 | 18.7383 | -20.8597 | 26.8701 | 13.7886 | 6.3640 |
| -2.5010 | -1.6981 | -15.7528 | -20.8394 | 36.7127 | -43.9380 | 4.0523 | 0.2833 |
| -1.2545 | 4.5706 | 22.5070 | 0.5628 | 5.7651 | -9.1108 | 2.5339 | -48.6327 |
| -0.4099 | 9.8925 | 7.3229 | 2.3845 | -26.8897 | 59.7817 | -10.4133 | 30.9166 |

← $C'(u,v) = C(u,v) \,./\, T(u,v)$

# Example

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 7 | 2 | -2 | -4 | 6 | 6 | 2 | -15 |
| -2 | 1 | 8 | -1 | 3 | -21 | 8 | -16 |
| -3 | -11 | -9 | 11 | -12 | 50 | 7 | 23 |
| -2 | 8 | 12 | 4 | -24 | 4 | -10 | 19 |
| -1 | 2 | -6 | -21 | 46 | 0 | 8 | -21 |
| 0 | 7 | -1 | 1 | -2 | -6 | 6 | 32 |
| -2 | 3 | 22 | 20 | -6 | 35 | 8 | -40 |
| -4 | 2 | -12 | -2 | 14 | -28 | -10 | -1 |

Differences between the input block and the reconstructed block

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 96 | 92 | 93 | 92 | 104 | 111 | 110 | 75 |
| 78 | 92 | 104 | 93 | 93 | 89 | 110 | 94 |
| 71 | 67 | 81 | 91 | 78 | 180 | 110 | 123 |
| 78 | 28 | 31 | 84 | 67 | 105 | 110 | 149 |
| 76 | 14 | 14 | 69 | 67 | 203 | 110 | 159 |
| 65 | 27 | 34 | 49 | 78 | 97 | 110 | 154 |
| 68 | 32 | 35 | 30 | 93 | 145 | 110 | 140 |
| 84 | 21 | 11 | 17 | 104 | 92 | 110 | 129 |

Reconstructed 8x8 pixels

+128

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| -144 | -231 | -220 | -176 | -120 | 0 | -51 | 0 |
| 12 | 72 | 84 | 76 | 0 | 0 | 0 | -55 |
| 14 | 39 | 32 | -24 | 40 | -57 | 0 | -56 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 18 | -22 | -37 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 100 | 0 | 0 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| -32.1959 | -36.1251 | -34.8882 | -36.0421 | -23.9488 | -16.5761 | -18.0312 | -52.8402 |
| -49.6080 | -36.4974 | -23.6556 | -35.2218 | -34.7727 | -38.6850 | -18.0312 | -33.5806 |
| -56.6210 | -61.3545 | -47.4893 | -36.5215 | -50.0801 | 52.4800 | -18.0312 | -4.5630 |
| -49.8443 | -100.4414 | -97.4515 | -43.7254 | -60.9040 | -22.7087 | -18.0312 | 20.5036 |
| -52.1854 | -114.4879 | -113.8391 | -58.5523 | -60.9040 | 75.3698 | -18.0312 | 31.2336 |
| -63.2879 | -101.3555 | -94.1572 | -78.7449 | -50.0801 | -30.6670 | -18.0312 | 25.9933 |
| -59.5857 | -96.3632 | -93.4991 | -98.4134 | -34.7727 | 16.8720 | -18.0312 | 12.1503 |
| -43.9653 | -106.7416 | -117.2741 | -110.5818 | -23.9488 | -36.0851 | -18.0312 | 1.1030 |

$C'(u,v) = C(u,v) .* T(u,v)$ → IDCT

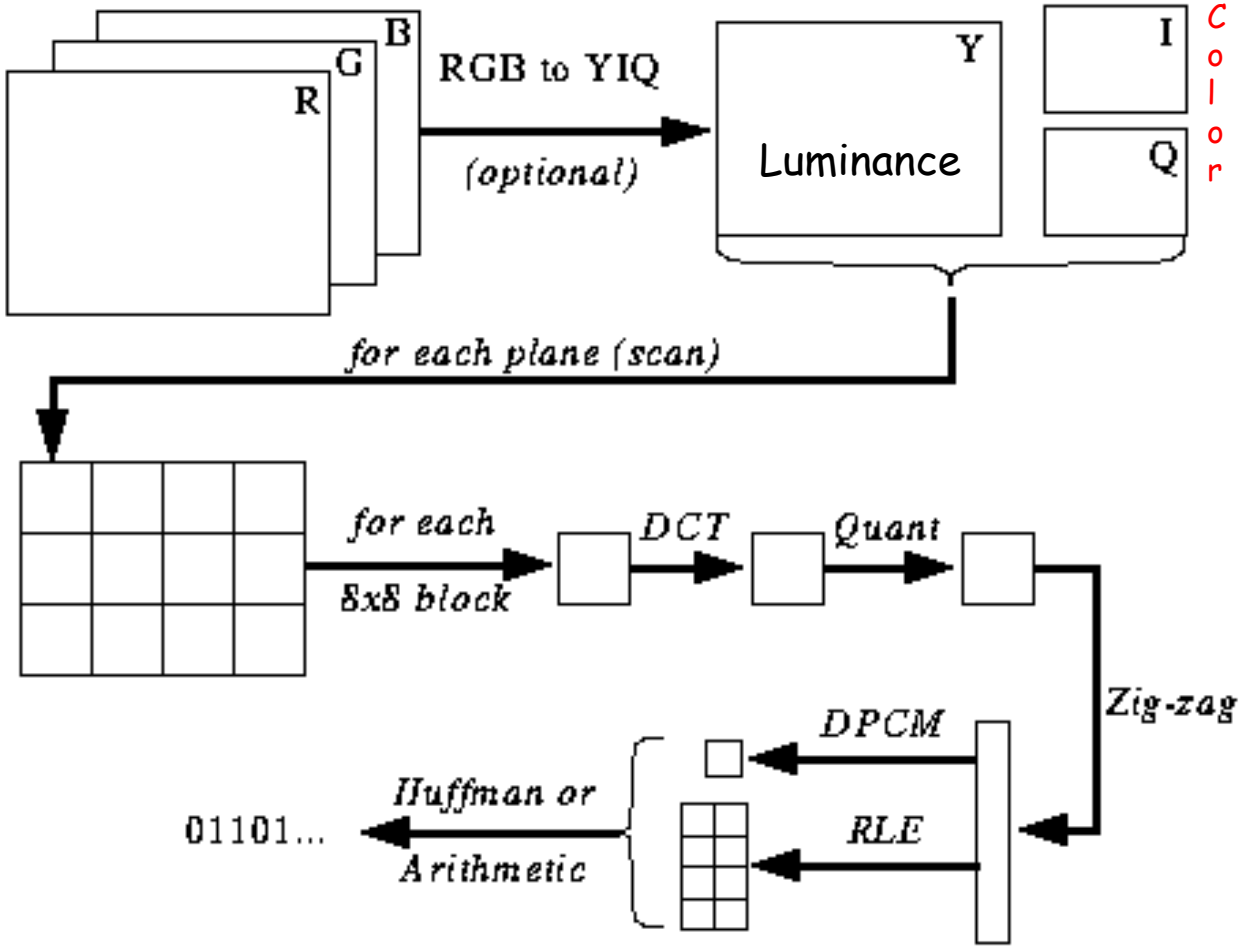
# JPEG “Quality”

- JPEG uses the term “quality” for its compression
  - Different quality factors correspond to different quantization tables
    - Lower Quality corresponds to larger values in the quantization table
  - Quality 100% is quantization table of T(u,v)’s 1
    - a little info lost from rounding

# JPEG

- Quantization is similar to blurring
- You also get a “blocking” effect from the 8x8 blocks
- JPEG Quantization Tables: Code for generating the Quantization tables used by JPEG
- For more information, https://en.wikipedia.org/wiki/JPEG

# Color Image Compression via JPEG




$$\begin{bmatrix} Y \\ I \\ Q \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ 0.596 & -0.275 & -0.321 \\ 0.212 & -0.523 & 0.311 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

# Compressor/Decompressor

- CODEC
  - Short for Compressor + Decompressor
- Engines for compression
- Different CODECs might give slightly different results
  - Some approximate FDCT/IDCT with integer math and/or lookup tables